

DOSSIER DE CONCEPTION

Document de travail — à valider avant développement

ÉTAPE 7

Trajets de confiance

Partager sa route à cinq personnes, et rentrer confirmé

Taxi et rendez-vous — arriver, confirmer, ou alerter



Périmètre Talky (Flutter) ■ Alanya-Backend

Statut Conception — aucune ligne de code écrite

Dépend de Listes de contacts (liste trust, migration 050)

Remplace —

Date 13 août 2026

Avant-propos

Alanya sait déjà partager *une* position : un message de type 5, une carte, un pin. C'est une photographie. Ce document décrit comment en faire un film utile — et surtout, comment le doubler d'une promesse qui tient même quand le film s'interrompt.

Une décision gouverne tout le reste, et il vaut mieux la lire avant le sommaire : **la garantie de sûreté est l'échéance, pas la trace**. La carte rassure ; c'est l'échéance qui protège. Elle vit côté serveur et ne dépend ni du GPS, ni de la batterie, ni du système d'exploitation. Trois conséquences en découlent, qui reviennent à chaque chapitre : perdre la position n'est pas un incident, iOS n'a pas à égaler Android, et une arrivée détectée ne clôt jamais un trajet toute seule.

Ce volet est aussi le premier à porter un risque de *détournement* : une fonctionnalité qui diffuse la position de quelqu'un peut servir à le surveiller. Le sujet n'est pas relégué en annexe — il a son chapitre dans chacune des deux parties.

Comment lire ce document. La **partie I** décrit la fonctionnalité telle que l'utilisateur et son cercle la voient. La **partie II** décrit comment on la construit. On peut valider la partie I sans lire la partie II.

Table des matières

Avant-propos	1
I Approche naïve	5
1 Le besoin	6
1.1 Ce qui existe déjà	6
1.2 Pourquoi le message ne convient pas au temps réel	6
1.3 Ce qu'on ajoute	6
1.4 La promesse, et ce qu'elle n'est pas	7
1.5 Les règles produit	7
2 Le cercle de confiance	8
2.1 Pourquoi réutiliser la liste Confiance	8
2.2 Le cercle est l'audience — en entier	8
2.3 Ce que le membre découvre	8
2.4 Ce que le membre peut refuser	8
3 Les écrans	10
3.1 Vue d'ensemble	10
3.2 Partir	11
3.3 Suivre	11
3.4 Côté membre	12
3.5 Décider et alerter	13
3.6 Après le trajet	14
3.7 Quand ça se dégrade	15
3.8 Points d'entrée	15
4 Les règles de vie d'un trajet	16
4.1 L'échéance, la grâce, et les trois relances	16
4.2 Le cercle est figé au départ	16
4.3 Quand quelqu'un bloque, quand le cercle se vide	16
4.4 Ce qu'on garde, et combien de temps	17
4.5 Ce que l'administration voit — et ne voit pas	17
4.6 Ce qui n'est pas dans cette étape	17
5 Ce qui pourrait mal tourner	18

5.1	Le détournement par un proche	18
5.2	La fatigue d'alerte	18
5.3	La fausse alerte	18
5.4	La batterie et la précision	18
II	Approche technique	20
6	La propriété de sûreté	21
6.1	Le désaccord à trancher en premier	21
6.2	Trois conséquences, à ne jamais perdre de vue	21
7	La machine à états	22
7.1	Les huit états	22
7.2	L'automate	23
7.3	Les transitions	24
7.4	Quatre invariants	24
7.5	Les cas limites	25
8	Modèle de données	26
8.1	Quatre tables	26
8.2	Pourquoi DECIMAL(9,6) et aucun index spatial	28
8.3	L'instantané de l'audience	28
8.4	Mémoire et base	29
9	Le fil des positions	30
9.1	Trois régimes, pilotés par la distance	30
9.2	Aucune de ces valeurs n'est écrite en dur	30
9.3	Décimation à trois étages	31
9.4	Détection d'arrivée	32
9.5	Coût réseau et charge serveur	32
10	L'échéance et l'escalade	34
10.1	L'escalade en trois temps	34
10.2	Les huit kind de jobs	34
11	API	36
11.1	REST — src/routes/trips.js	36
11.2	Socket.IO — src/socket/handlers/trips.js	37
11.3	Le verrou d'appareil	37
12	Le message de type 9	38
12.1	Pourquoi un type de message	38

12.2 L'état dans le message, le mouvement dans le socket	38
12.3 Compatibilité descendante	38
13 Notifications	39
14 Client Flutter	40
14.1 Cache local — Drift 23 → 24	40
14.2 Le service de localisation en avant-plan (Android)	40
14.3 Permissions à ajouter	41
14.4 iOS — ce qui est garanti, et ce qui ne l'est pas	41
14.5 Interface	41
15 Vie privée, coercition et administration	43
15.1 Base légale et minimisation	43
15.2 Rétention — le tableau fait foi	43
15.3 Ce que l'administration voit — et ce qu'on refuse de construire	43
15.4 Détournement et coercition	44
15.5 Le blocage, et le cercle qui se vide	44
16 Impact fichier par fichier	46
16.1 Backend	46
16.2 Mobile	47
16.3 Administration	47
17 Vérification	48
17.1 Backend	48
17.2 Client	48
17.3 Recette terrain — ce qui ne se teste que dehors	48
17.4 Points ouverts	49

PARTIE I

Approche naïve

CHAPITRE 1

Le besoin

1.1 Ce qui existe déjà

Alanya sait partager **une** position. Depuis le menu des pièces jointes d’une conversation, on ouvre une carte, on déplace le pin, on envoie. Le destinataire reçoit une vignette ; un appui l’ouvre dans son application de cartographie.

Un partage ponctuel, complet et fonctionnel. Le sélecteur est `lib/screens/chats/location_picker_screen.dart` : carte OpenStreetMap, pin fixe au centre, et une chaîne de repli soignée — service désactivé, permission refusée, `getCurrentPosition` trop lent, dernière position connue. Le message est un message ordinaire de **type 5**, dont le **content** est un JSON `{lat, lng, name, address}` produit par `lib/core/utils/location_payload.dart`, avec un géocodage inverse Nominatim en meilleur effort.

Ce qui n’existe pas : le *mouvement*. Une position envoyée est une photographie. Elle ne dit pas si la personne avance, si elle s’est arrêtée, ni si elle est arrivée.

1.2 Pourquoi le message ne convient pas au temps réel

L’idée la plus immédiate serait de renvoyer un message de type 5 toutes les trente secondes. Elle ne tient pas :

- la conversation deviendrait illisible — des dizaines de vignettes pour un seul trajet ;
- chaque envoi remonterait la conversation en tête de liste, rallumerait le compteur de non-lus et déclencherait une notification ;
- l’historique serait impossible à tenir : on ne saurait pas dire où commence et où finit un trajet, ni le supprimer sans supprimer des messages ;
- et surtout, rien là-dedans ne *surveille* l’arrivée. Or c’est cela, la demande.

1.3 Ce qu’on ajoute

Un objet à part : le **trajet**. On le démarre, on annonce où l’on va et quand on compte arriver, ses proches le suivent, et il se termine par une confirmation — ou par une alerte.

La phrase à retenir. La promesse n’est pas « vos proches voient où vous êtes ». Elle est : « **si vous ne confirmez pas votre arrivée, vos proches seront prévenus** ».

La carte est un confort. L’échéance est le contrat. C’est une distinction de conception, pas une nuance de vocabulaire : elle décide de tout ce qui suit, à commencer par le fait qu’une perte de signal n’est jamais une alerte.

1.4 La promesse, et ce qu'elle n'est pas

Ce n'est pas un traceur	Il n'existe aucun partage permanent, aucune récurrence, aucun interrupteur « toujours actif ». Un trajet est fini, et démarré à la main.
Ce n'est pas un service d'urgence	Le SOS prévient le cercle de confiance. Il n'appelle ni la police, ni les secours, et l'écran le dit.
Ce n'est pas une demande	Personne ne peut réclamer la position de quelqu'un. Seul le propriétaire démarre, et seul lui arrête.
Ce n'est pas un historique partagé	Les membres du cercle voient le trajet <i>pendant</i> qu'il a lieu. Ils n'ont accès à aucun trajet passé.

1.5 Les règles produit

Qui reçoit	Tout le cercle de confiance , c'est-à-dire la liste système <i>Confiance</i> , plafonnée à cinq. Pas de sous-sélection au départ.
Un seul à la fois	Un utilisateur n'a qu'un trajet ouvert. Repartir, c'est un nouveau trajet.
L'arrivée ne clôt rien	Le GPS <i>propose</i> , l'humain <i>dispose</i> . Une destination atteinte pose la question ; elle n'y répond pas.
Seul le propriétaire clôt	Un membre peut dire « j'ai vu » et appeler. Il ne peut pas décider que quelqu'un est arrivé.
Dix minutes de grâce	Entre l'échéance et l'alerte. Avec trois relances, adressées au propriétaire seul.
Une alerte ne se retire pas	Elle se <i>résout</i> . Le journal garde la trace de ce qui est parti, et à qui.
Silence n'est pas danger	Un GPS perdu s'affiche en gris, jamais en rouge. Le rouge est réservé à l'alerte et au SOS.
Douze heures au maximum	Passé ce plafond, le trajet demande à être clos.
Navigation	Aucun sixième onglet. La barre est codée en dur à cinq onglets (<code>lib/screens/home/glass_nav_bar.dart</code>).

CHAPITRE 2

Le cercle de confiance

2.1 Pourquoi réutiliser la liste Confiance

Le volet 2 a créé quatre listes système, dont une : *Confiance*, seule à porter un plafond — cinq membres. Elle attendait un usage ; c’est celui-ci.

Le plafond existe déjà en base. `src/utils/defaultContactLists.js` sème la liste `kind = 'trust'` avec `member_limit = 5` et la couleur `#B7791F`, à l’inscription et en rattrapage à chaque lecture des listes. Le contrôleur refuse le sixième membre en 409 `LIST_MEMBER_LIMIT`. Rien n’est à créer côté audience.

Ne pas inventer une seconde notion d’audience est aussi la position déjà tranchée au volet 2, qui a refusé de fusionner listes personnelles et audiences de diffusion. Ici, on va dans le même sens : on *réutilise* la liste, on n’en fabrique pas une autre.

2.2 Le cercle est l’audience — en entier

Démarrer un trajet prévient les cinq. Il n’y a pas de case à cocher, pas d’écran de sélection, pas de « partager à trois sur cinq ».

Ce que ce choix achète. Un appui de moins, et une promesse énonçable en une phrase — « mon cercle est prévenu » — au lieu d’une liste variable dont on ne se souvient plus le lendemain. C’est aussi ce qui permet de tenir l’objectif de vingt secondes entre l’envie de partir et le départ effectif. Le corollaire : on compose son cercle **à froid**, dans ses listes de contacts, pas au moment de partir. Cette composition ne notifie personne.

2.3 Ce que le membre découvre

Une règle du volet 2 est mise en défaut — assumons-le. Les listes de contacts sont aujourd’hui **strictement privées** : « personne ne sait dans quelle liste il est rangé, ni même qu’il l’est » (`contenu/listes-naive.tex`). Le premier trajet partagé **rompt cette règle** : en recevant la carte, un proche apprend qu’il fait partie du cercle de confiance.

On ne peut pas l’éviter — prévenir quelqu’un sans qu’il sache qu’il a été choisi n’a pas de sens. On l’assume donc, et l’écran du membre le dit explicitement : « *Awa vous a choisi comme personne de confiance* ». Ce qui reste privé : les *autres* listes, et le fait d’être *sorti* du cercle.

Le membre voit combien de personnes suivent, jamais qui. Cela rassure — « je ne suis pas seul à veiller » — sans exposer le carnet d’adresses de quelqu’un d’autre.

2.4 Ce que le membre peut refuser

Recevoir un trajet n’est pas une obligation. Le membre peut quitter un trajet en cours, et un réglage général permet de ne plus être destinataire de trajets. Dans les deux cas, le propriétaire lit

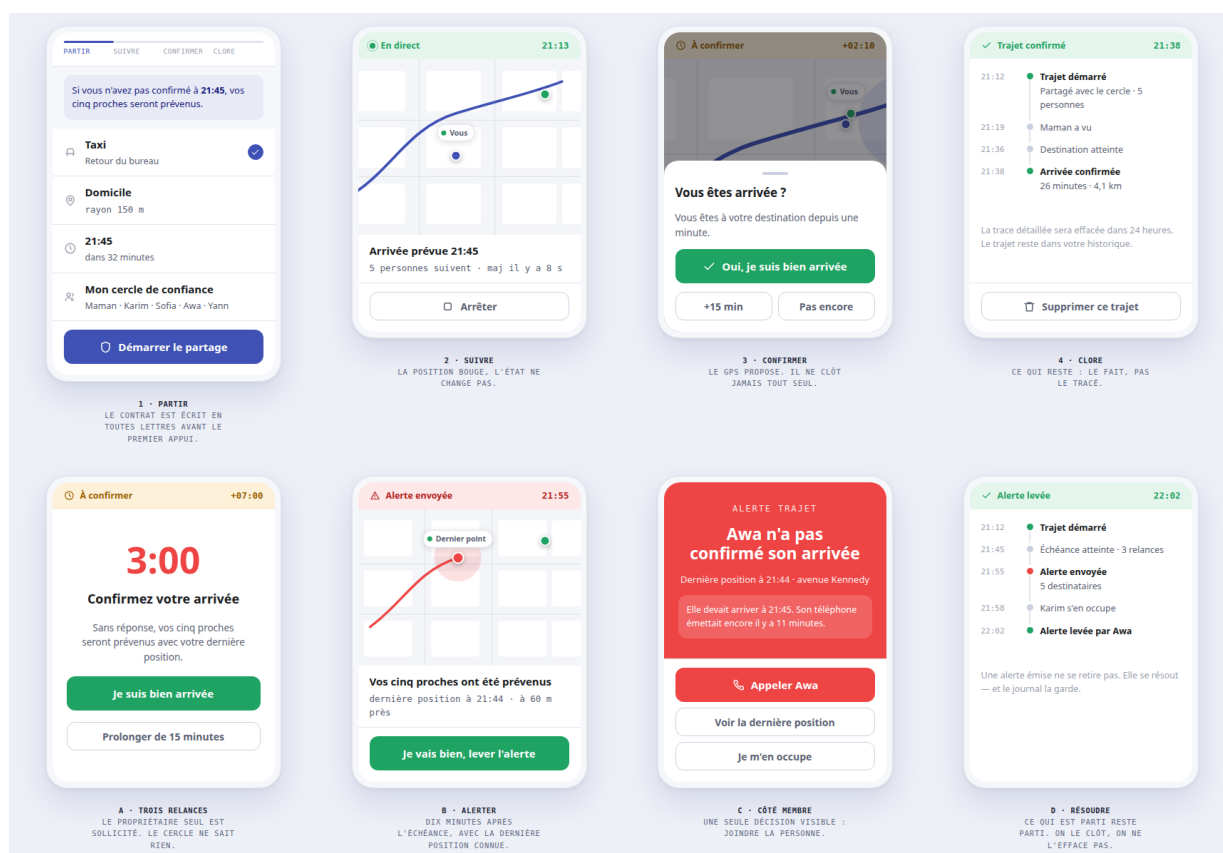
« un membre n'est plus destinataire », sans motif.

CHAPITRE 3

Les écrans

3.1 Vue d'ensemble

Deux chemins partent de la même feuille de départ : celui où l'on confirme, et celui où le silence déclenche l'alerte.

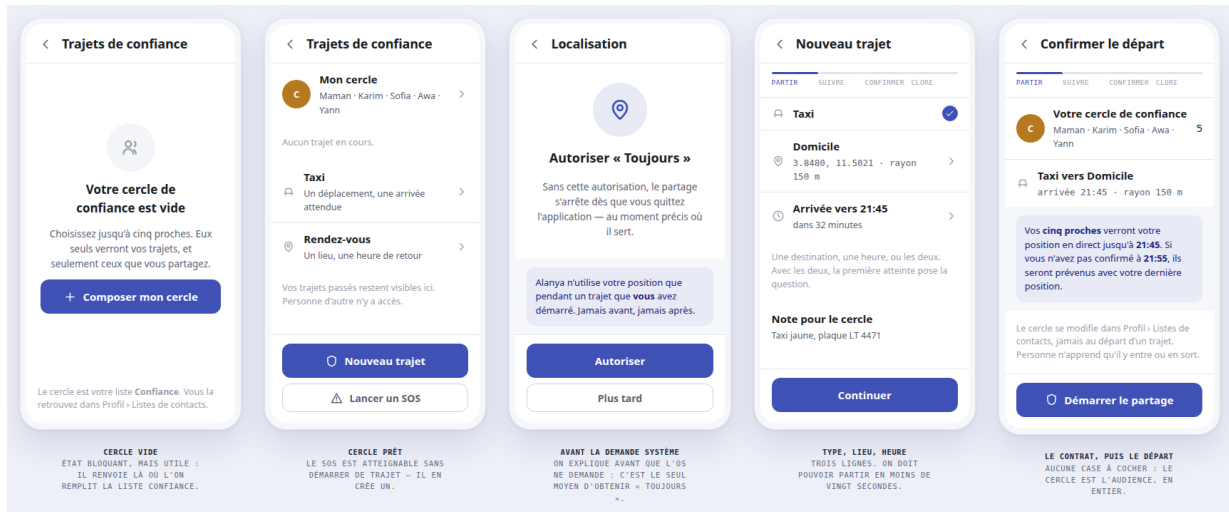


Maquette interactive. claude.ai/code/artifact/fd08532c

Source versionnée : [docs/architecture/maquettes/trajets-parcours.html](https://docs.architecture/maquettes/trajets-parcours.html)

Les quatre temps — **partir**, **suivre**, **confirmer**, **clore** — sont rappelés en tête de chaque écran du propriétaire par une rampe de quatre segments. C'est la seule chose qui ne change jamais de place.

3.2 Partir



Maquette interactive. claude.ai/code/artifact/39bf86b6

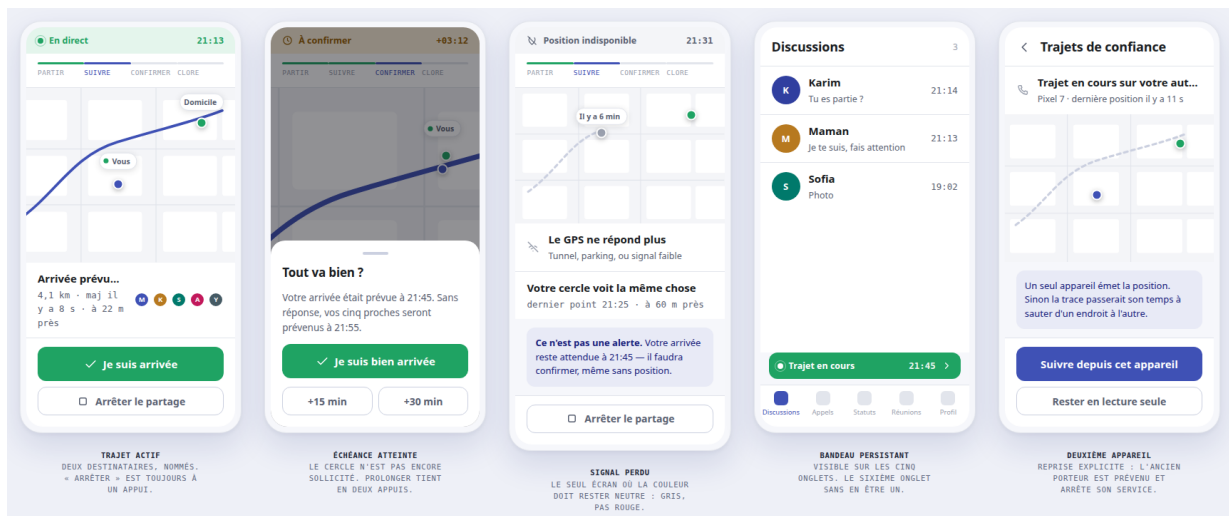
Source versionnée : [docs/architecture/maquettes/trajets-depart.html](https://docs.architecture/maquettes/trajets-depart.html)

Trois lignes suffisent : le type (taxi ou rendez-vous), le lieu, l'heure. On peut n'en donner qu'une des deux dernières — avec les deux, la première atteinte pose la question.

Le dernier écran affiche **le contrat en toutes lettres** : « *Vos cinq proches verront votre position jusqu'à 21 :45. Si vous n'avez pas confirmé à 21 :55, ils seront prévenus avec votre dernière position.* » C'est le seul élément de tout le parcours qui garantit que la personne a compris ce qu'elle déclenche. Les deux heures y sont écrites, pas déduites.

Deux écrans ne se voient qu'une fois : la composition du cercle, quand il est vide, et l'explication de la permission de localisation — posée *avant* que le système ne la demande, parce que c'est le seul moyen d'obtenir « Toujours autoriser ».

3.3 Suivre



Maquette interactive. claude.ai/code/artifact/32f2cae7

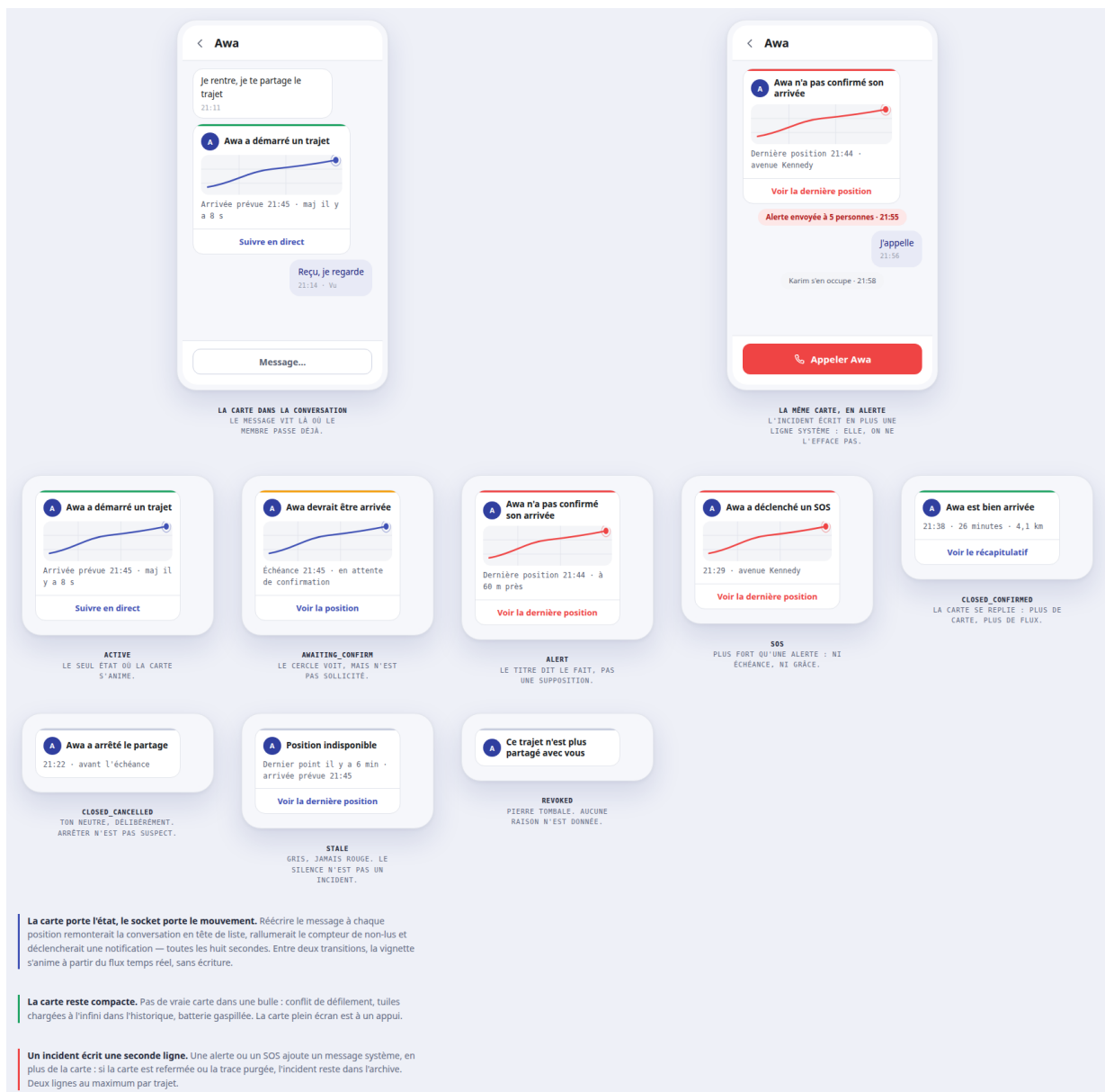
Source versionnée : [docs/architecture/maquettes/trajets-en-cours.html](https://docs.architecture/maquettes/trajets-en-cours.html)

Pendant tout le trajet, un **bandeau** se pose au-dessus de la barre de navigation. Il est visible sur les cinq onglets, affiche l'heure d'arrivée, et s'ouvre d'un appui.

Le bandeau tient dans un espace déjà réservé. `lib/screens/home/glass_nav_bar.dart` construit sa rangée par `List.generate(5, ...)` et réserve `kGlassNavBarSpace = 96` sous le contenu. Le bandeau s'insère là. Aucun sixième onglet, aucune refonte de navigation.

« Arrêter le partage » est toujours à un appui, y compris depuis la notification système. Ce n'est pas une commodité : si arrêter demandait d'ouvrir l'application et de naviguer, arrêter deviendrait coûteux — donc reprochable par quelqu'un qui regarde par-dessus l'épaule.

3.4 Côté membre



Maquette interactive. claude.ai/code/artifact/065db5f1

Source versionnée : [docs/architecture/maquettes/trajets-membre.html](https://docs.architecture/maquettes/trajets-membre.html)

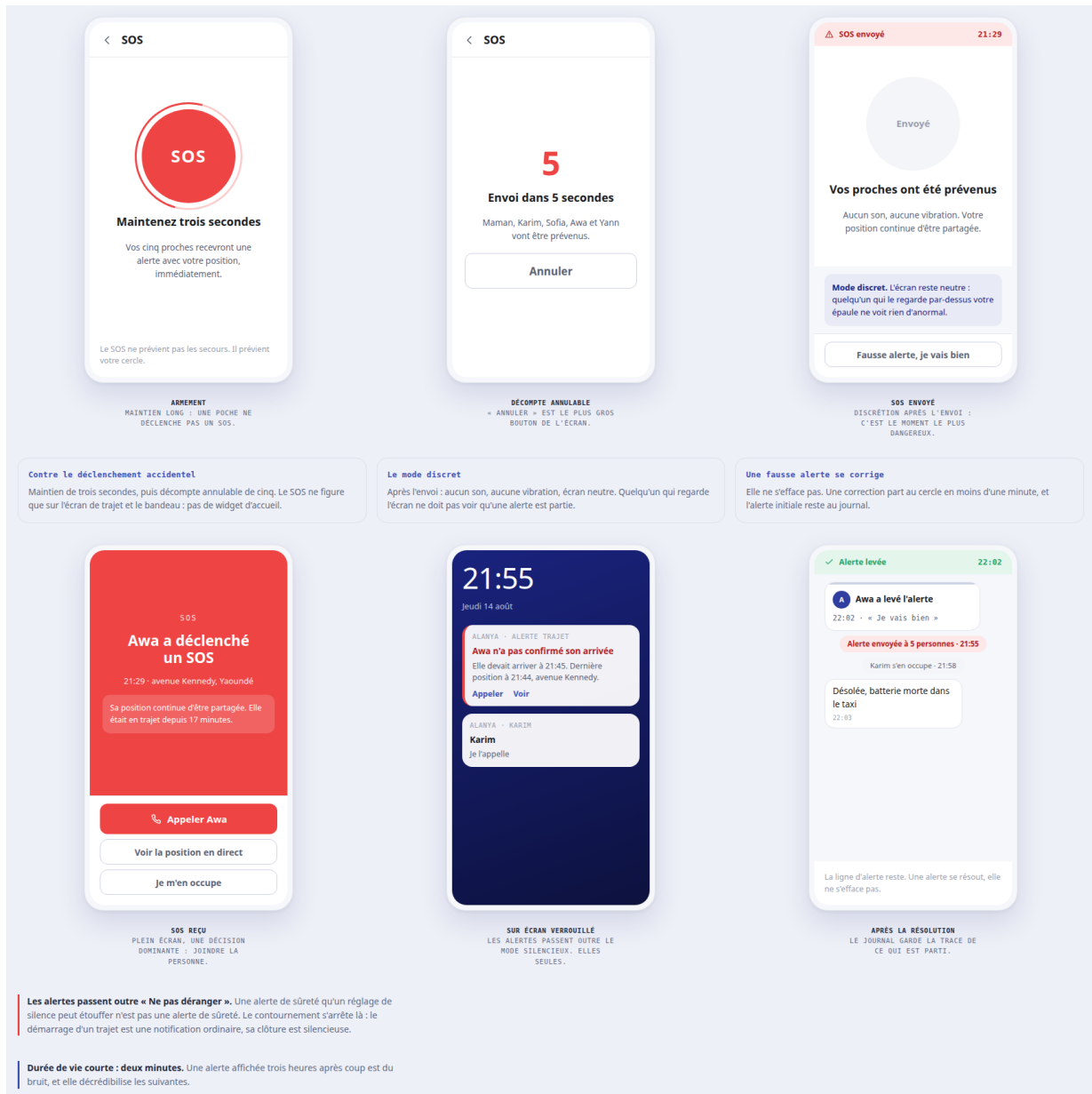
Le trajet arrive chez le proche **dans la conversation**, là où il parle déjà avec la personne. C'est la

seule surface qu'on est certain qu'il verra : elle reçoit déjà la notification, elle survit au redémarrage, elle reste dans l'archive.

Un **seul message** par trajet, qui change d'état sur place — il ne remonte pas la conversation à chaque position. Il reste compact : avatar, ce qui se passe, l'heure d'arrivée, et un bouton « Suivre ». La vraie carte est plein écran, à un appui.

Le membre a exactement deux gestes : dire qu'il a vu, et appeler.

3.5 Décider et alerter



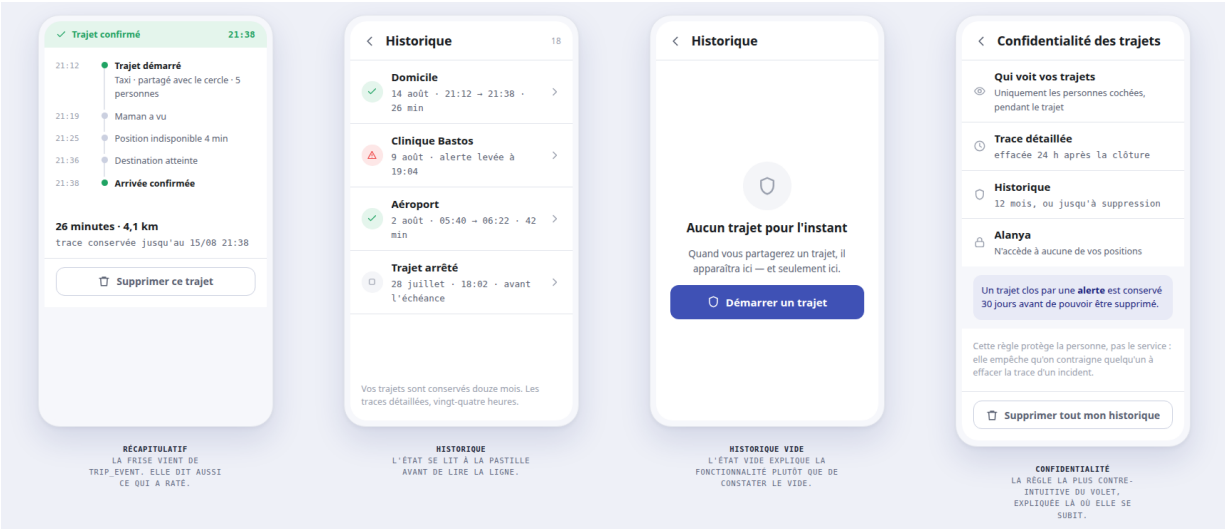
Maquette interactive. claude.ai/code/artifact/91c2ad6f

Source versionnée : docs/architecture/maquettes/trajets-alerte.html

Le **SOS** est accessible sans trajet en cours — il en crée un, sans destination ni heure, immédiatement en alerte. Il demande deux gestes pour partir : un maintien de trois secondes, puis un décompte annulable de cinq. Après l'envoi, l'écran redevient neutre : aucun son, aucune vibration. C'est le moment le plus dangereux, et il ne doit rien laisser paraître.

Une fausse alerte se *corrige* en un appui, en moins d’une minute. Elle ne s’efface pas.

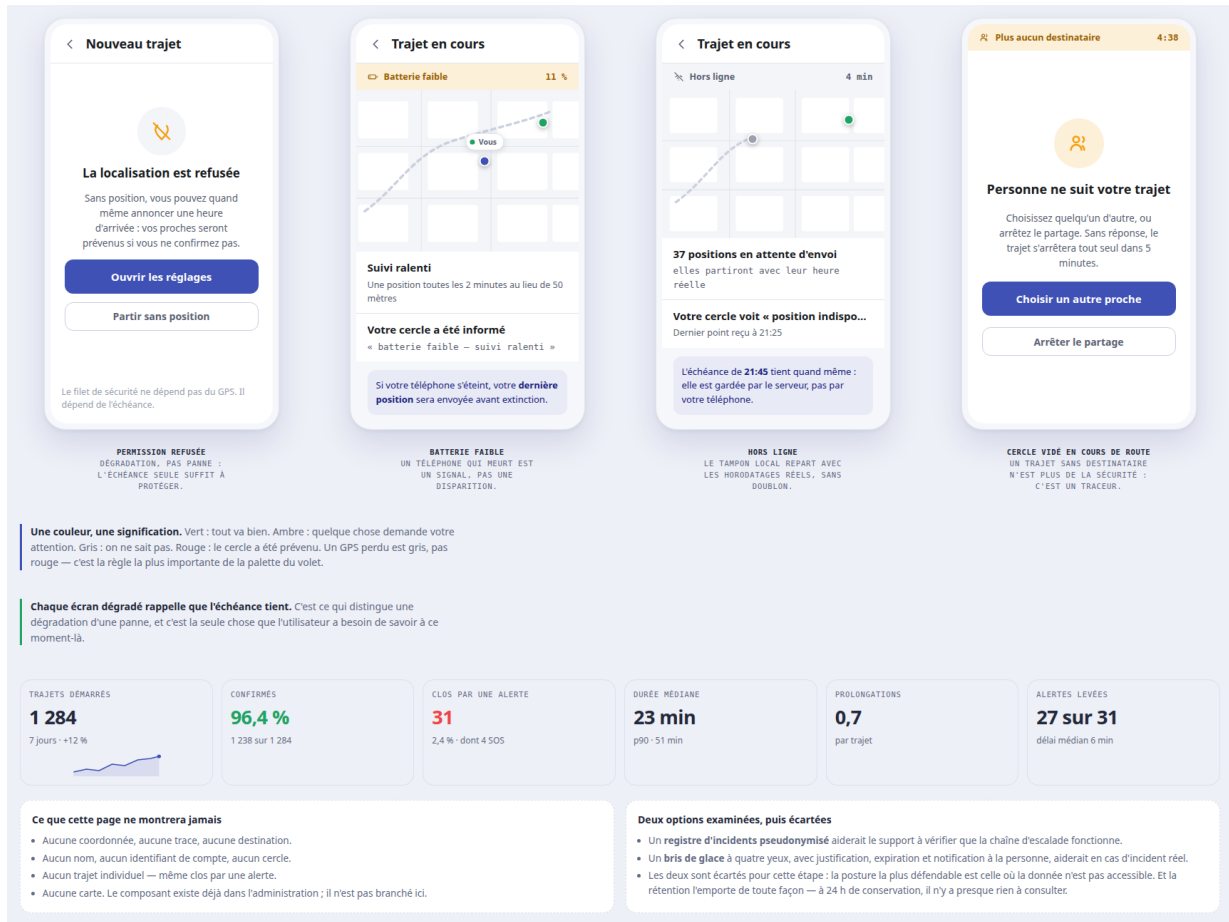
3.6 Après le trajet



Maquette interactive. claude.ai/code/artifact/048b398f
Source versionnée : docs/architecture/maquettes/trajets-fin-historique.html

Le récapitulatif rejoue la frise du trajet — y compris ce qui a raté, comme les quatre minutes sans signal. L’historique liste les trajets passés, avec leur issue lisible à la pastille avant même de lire la ligne.

3.7 Quand ça se dégrade



Maquette interactive. claude.ai/code/artifact/556526a1

Source versionnée : [docs/architecture/maquettes/trajets-erreurs-admin.html](https://docs.architecture/maquettes/trajets-erreurs-admin.html)

Aucun de ces quatre écrans n'est rouge. Chacun rappelle que **l'échéance tient** — ce qui est la seule chose que l'utilisateur ait besoin de savoir à ce moment-là, et ce qui distingue une dégradation d'une panne.

3.8 Points d'entrée

1. Le profil, entrée « Trajets de confiance », à côté de « Listes de contacts » (`lib/screens/profile/profile_screen.dart`). C'est l'entrée canonique : démarrer, voir le trajet en cours, relire l'historique.
2. Le menu des pièces jointes d'une conversation, où « Position » devient un choix à deux entrées : *position actuelle* — le type 5 existant — ou *trajet de confiance*. C'est là que vit déjà le réflexe « je partage où je suis ».
3. Le bandeau persistant, tant qu'un trajet est ouvert.

CHAPITRE 4

Les règles de vie d'un trajet

4.1 L'échéance, la grâce, et les trois relances

Moment	Qui est sollicité	Ce qui se passe
$T - 5 \text{ min}$	Le propriétaire, en silence	« Vous arrivez bientôt ? »
T	Le propriétaire, notification prioritaire	Le trajet passe « à confirmer »
$T + 3, T + 7 \text{ min}$	Le propriétaire, relances	Décompte visible
$T + 10 \text{ min}$	Le cercle	Alerte, avec la dernière position connue

Pourquoi dix minutes, et non cinq. En deçà de cinq minutes, chaque embouteillage devient une alerte — et le cercle apprend à ne plus les ouvrir. C'est le véritable mode de défaillance de ce genre de produit : pas l'alerte manquée, mais l'alerte qu'on ignore. Au-delà d'un quart d'heure, le délai n'a plus d'utilité dans un incident réel.

Prolonger tient en deux appuis, sans limite de nombre, et le cercle en est informé : « *Awa a prolongé de 15 minutes* ». Cela rassure sans alerter.

4.2 Le cercle est figé au départ

Les cinq destinataires sont photographiés au démarrage. Modifier sa liste Confiance pendant un trajet ne change pas le trajet en cours : si l'alerte part, on doit pouvoir dire exactement qui a été prévenu.

4.3 Quand quelqu'un bloque, quand le cercle se vide

Le blocage l'emporte sur tout. Un membre qui bloque le propriétaire cesse d'être destinataire — alertes comprises. Le propriétaire lit « un membre n'est plus destinataire », jamais « X vous a bloqué » : l'application ne révèle pas un blocage.

Un trajet sans destinataire ne continue pas. S'il ne reste plus personne, le trajet ne doit pas se poursuivre en silence : il ne serait plus une fonctionnalité de sécurité, mais un traceur personnel. Le propriétaire reçoit une invite immédiate ; sans réponse au bout de cinq minutes, le trajet se clôt tout seul.

4.4 Ce qu'on garde, et combien de temps

	Trajet normal	Après une alerte	Pourquoi
La trace détaillée	24 h après clôture	30 jours	Un registre permanent des déplacements de tous les utilisateurs n'est justifié par aucun besoin. Après un incident, la trace peut servir de preuve.
Le trajet lui-même	12 mois	12 mois	L'historique se contente du départ, de l'arrivée, de la durée et de l'issue.

Le verrou de trente jours protège la personne, pas le service. Un trajet clos par une alerte ne peut pas être supprimé avant trente jours. C'est délibéré : personne ne doit pouvoir contraindre un proche à effacer la trace d'un incident dans la minute. L'écran de confidentialité l'explique, pour que la règle ne se découvre pas par un refus.

4.5 Ce que l'administration voit — et ne voit pas

Une seule page : des **compteurs agrégés**. Trajets démarrés, taux de confirmation, nombre d'alertes, durée médiane. Aucune coordonnée, aucun nom, aucune destination, aucun trajet individuel.

Deux options ont été examinées puis écartées — un registre d'incidents pseudonymisé, et un accès exceptionnel à la dernière position sous double validation. Elles aideraient le support. Elles sont écartées parce que la posture la plus défendable est celle où la donnée n'est pas accessible ; et parce qu'à vingt-quatre heures de conservation, il n'y aurait de toute façon presque rien à consulter.

4.6 Ce qui n'est pas dans cette étape

Déclenchement matériel du SOS par le bouton d'alimentation ; appel automatique des secours ; partage à quelqu'un qui n'est pas dans le cercle, par lien ; trajets récurrents. Aucun de ces points ne demanderait de changer le modèle de données.

CHAPITRE 5

Ce qui pourrait mal tourner

Une fonctionnalité qui diffuse la position de quelqu'un mérite qu'on écrive ses risques avant son code. Les quatre suivants sont les plus sérieux, et chacun a sa contre-mesure dans la conception — pas dans un réglage.

5.1 Le détournement par un proche

C'est le risque qui peut tuer la fonctionnalité : un conjoint qui exige un trajet à chaque sortie. Sept garde-fous, tous structurels :

1. aucune demande de position n'existe — le propriétaire est toujours l'émetteur ;
2. tout trajet est fini, et démarré à la main ;
3. la notification persistante **nomme les destinataires** et ne peut être masquée ;
4. les membres n'ont **aucun historique** — « montre-moi où tu étais mardi » n'est pas une fonctionnalité ;
5. l'audience se compose à froid, en silence, hors du moment de partir ;
6. « Arrêter le partage » est toujours à un appui, et un arrêt anticipé n'envoie qu'un message neutre ;
7. un trajet clos par une alerte ne s'efface pas avant trente jours.

Ce sont des atténuations, pas une solution. Aucune de ces mesures n'empêche de contraindre quelqu'un à démarrer un trajet. Elles rendent la contrainte moins rentable et moins durable. Il faut l'écrire tel quel — prétendre le contraire serait la plus dangereuse des erreurs de conception.

5.2 La fatigue d'alerte

Une alerte qu'on n'ouvre plus ne protège personne. D'où les trois relances avant que le cercle n'entende quoi que ce soit, la prolongation en deux appuis, et une graduation stricte des notifications : le démarrage est ordinaire, la clôture est silencieuse, seules l'alerte et le SOS sont bruyants.

5.3 La fausse alerte

Un SOS parti d'une poche détruirait la confiance du cercle plus sûrement qu'une alerte manquée. Maintien de trois secondes, décompte annulable de cinq, bouton absent de l'écran d'accueil, et correction poussée au cercle en moins d'une minute.

5.4 La batterie et la précision

Le suivi n'est pas continu : il se déclenche à la distance parcourue, avec un battement régulier qui distingue *immobile* de *traceur éteint*. Sous quinze pour cent, il ralentit et le cercle en est informé.

Quant à la précision, elle est dite plutôt que masquée : en ville, on est souvent à soixante mètres près, et l'écran l'affiche. C'est aussi pourquoi l'arrivée se détecte dans un rayon de cent cinquante mètres, tenu pendant une minute, à faible vitesse — passer devant sa rue sans s'arrêter est le faux positif le plus banal.

PARTIE II

Approche technique

CHAPITRE 6

La propriété de sûreté

6.1 Le désaccord à trancher en premier

Une fonctionnalité de suivi de trajet peut se construire autour de deux promesses différentes, et elles n'ont pas le même coût ni la même solidité :

1. « **vos proches voient où vous êtes** » — la promesse est la *trace* ;
2. « **si vous ne confirmez pas, vos proches sont prévenus** » — la promesse est l'*échéance*.

La première dépend du GPS, de la batterie, du réseau, du système d'exploitation et du bon vouloir d'iOS en arrière-plan. Elle tombe en panne régulièrement, et une fonctionnalité de sûreté qui tombe en panne régulièrement est pire qu'absente : on lui fait confiance.

La seconde ne dépend que d'une ligne dans `job_queue` et d'une horloge serveur.

Le postulat du volet. La garantie de sûreté, c'est l'*échéance* — pas la *trace*. La *trace* est un confort : elle rassure pendant, et elle aide à retrouver quelqu'un après. L'*échéance* est le contrat. Tout le reste du dossier découle de là.

6.2 Trois conséquences, à ne jamais perdre de vue

Conséquence 1 — perdre la position n'est pas un incident. Un téléphone dans un tunnel, un GPS coupé, une application tuée par le système : ce sont des *informations*, affichées honnêtement (« dernier point il y a 4 minutes »), et elles ne déclenchent jamais d'alerte à elles seules. Confondre « je ne vous vois plus » et « il vous est arrivé quelque chose » est le plus court chemin vers un cercle qui n'ouvre plus les notifications.

Conséquence 2 — iOS n'a pas à égaler Android. On ne peut pas garantir un flux de positions continu sur iOS en arrière-plan, et il est inutile de le promettre. Le filet tient quand même, parce qu'il est côté serveur. Le dossier dit ce qui est garanti et ce qui ne l'est pas, plutôt que d'annoncer une parité qui ne sera pas tenue.

Conséquence 3 — on ne clôt jamais automatiquement sur une arrivée détectée. Le GPS *propose*, l'humain *dispose*. Une détection d'arrivée pose la question ; seule la personne y répond. Le coût d'un faux positif devient une question à balayer, jamais un filet rompu.

Le filet dépend d'un drapeau d'environnement. `startJobWorker()` est conditionné par `BROADCAST_WORKER_ENABLED !== 'true'` (`src/services/jobQueue.js`). Aujourd'hui, si le drapeau est absent, la file ne tourne pas et seuls les diffusions et le rattrapage de bienvenue en pâtissent — fâcheux, pas grave. Avec les trajets, **aucune échéance ne se déclenche plus : on promet un filet qui n'existe pas**. Deux mesures dans le même lot : renommer le drapeau en `JOB_WORKER_ENABLED`, et refuser de servir **POST** `/api/trips` si le worker n'a pas démarré. Un refus explicite vaut mieux qu'une promesse silencieuse.

CHAPITRE 7

La machine à états

7.1 Les huit états

État	Nature	Sens
active	ouvert	Partage en cours, les positions circulent.
awaiting_confirm	ouvert	La question est posée. Le partage continue.
alert	ouvert	Grâce écoulee sans réponse. Le cercle est prévenu.
sos	ouvert	Alerte explicite. Strictement plus fort que <code>alert</code> .
closed_confirmed	terminal	La personne a confirmé son arrivée.
closed_cancelled	terminal	Arrêté par le propriétaire avant toute échéance.
closed_expired	terminal	Plafond dur atteint après une alerte.
closed_unwatched	terminal	Plus aucun destinataire.

Pourquoi des libellés anglais dans le schéma. Le reste de la base fait de même — `platform` `ENUM('android','ios','web','unknown')` (`migrations/020_user_push_devices.sql`), `kind = 'trust'` (`migrations/050_contact_list_kind.sql`). Les libellés français sont l'affaire de la couche de présentation et de `lib/110n/`, jamais celle du schéma.

ENUM plutôt qu'un entier — et le contre-exemple est dans la base. À taille de stockage identique — un octet dans les deux cas — l'entier ne survit pas à la relecture. `message.type` est un `SMALLINT` documenté par un commentaire SQL : `COMMENT '0=text 1=image 2=video 3=audio 4=file 5=location'` (`migrations/001_initial_schema.sql`). Ce commentaire s'arrête à 5, alors que les types **6, 7 et 8 existent** — message système, carte de contact, appel à l'action de bienvenue. Le commentaire a pourri ; la base ne sait plus ce qu'elle contient, et un dump de production ne se lit plus sans le code sous les yeux.

Un `ENUM` ne peut pas pourrir : la liste des valeurs *est* le schéma, elle sort avec `SHOW CREATE TABLE`, et le serveur refuse ce qui n'y figure pas. C'est déjà le choix retenu partout où la base nomme un état : `appState`, `block_type`, `login_method`, `previewMode`, et le `status` des diffusions.

La règle appliquée ici, et ses trois pièges. Ensemble fermé, qui ne change que par **décision de conception** → `ENUM : trip.state` — c'est une machine à états, y ajouter un état *doit* coûter une migration, c'est une garantie et non une gêne — ainsi que `trip.kind` et `trip_watcher.state`.

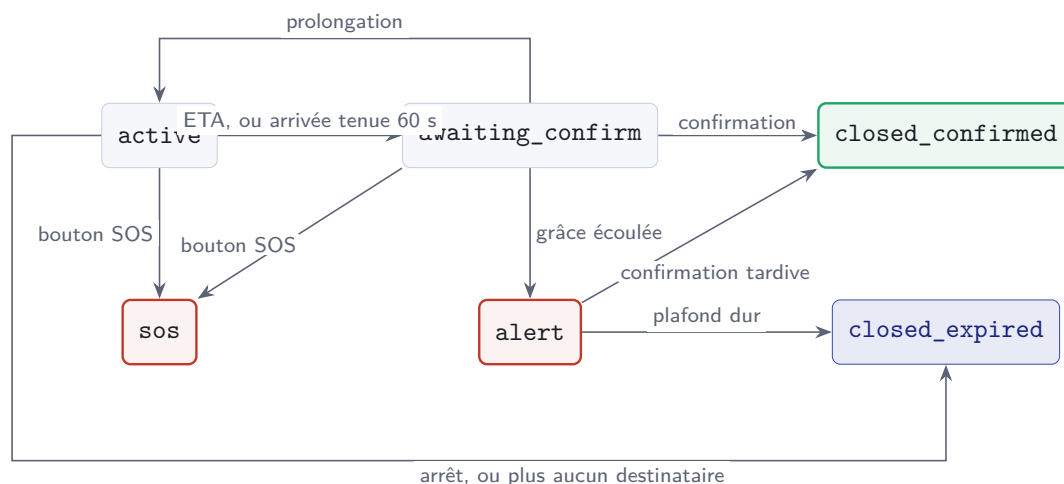
Ensemble ouvert, écrit depuis de nombreux endroits → `VARCHAR : trip_event.kind`, qui gagnera des valeurs au fil des versions, comme `job_queue.kind` avant lui.

Entier → seulement si la valeur est *ordinaire* et qu'on la compare avec `<` ou `>`. Aucune colonne de ce volet n'est dans ce cas.

Les trois pièges de l'`ENUM`, à connaître avant d'écrire la migration :

1. `WHERE state = 1` ne compare pas au texte « 1 » : il désigne la **première valeur déclarée**. On compare toujours à une chaîne.
2. `ORDER BY state` trie par ordre de *déclaration*, pas alphabétiquement. On déclare donc les valeurs dans l'ordre du cycle de vie — ici, c'est un avantage.
3. Ajouter une valeur **en fin de liste** se fait sans reconstruire la table, tant qu'on reste sous 255 valeurs et donc sur un octet. L'insérer au milieu, la renommer ou réordonner force au contraire une reconstruction complète. On ajoute toujours à la fin.

7.2 L'automate



Les deux états terminaux `closed_cancelled` et `closed_unwatched` partagent la même arête de sortie sur le schéma ; ils diffèrent par leur `close_reason` et par ce qui est notifié au cercle.

7.3 Les transitions

De	Vers	Qui	Déclencheur
—	active	client	POST /api/trips
—	sos	client	POST /api/trips/sos, aucun trajet ouvert
active	awaiting_confirm	job / serveur	trip_eta_due, ou rayon tenu 60 s à l'ingestion d'un point
awaiting_confirm	active	client	prolongation
awaiting_confirm	closed_confirmed	client	confirmation
awaiting_confirm	alert	job	trip_grace_expired
alert	closed_confirmed	client	confirmation tardive
ouvert	sos	client	bouton SOS après décompte
alert / sos	closed_expired	job	trip_max_duration
active / awaiting_confirm	closed_cancelled	client	arrêt
tout ouvert	closed_unwatched	job	trip_no_watcher

7.4 Quatre invariants

1. **Seul le propriétaire clôt.** Un destinataire ne peut qu'acquitter (**POST** /api/trips/:id/seen), ce qui s'inscrit dans `trip_event` et remonte au propriétaire (« Maman a vu »). Personne d'autre ne décide qu'une personne est arrivée.
2. **Une alerte émise ne se rétracte jamais.** Confirmer après une alerte émet un événement de résolution ; cela n'efface pas l'alerte, ni chez les destinataires, ni dans le journal.
3. **Terminal est terminal.** Repartir, c'est un nouveau trajet.
4. **Un seul trajet ouvert par utilisateur** — 409 TRIP_ALREADY_ACTIVE.

7.5 Les cas limites

Situation	Comportement
Application tuée	Le service en avant-plan tient le flux. Si le système tue quand même, le serveur constate la péremption ($3 \times$ battement + 30 s), pose <code>stale = 1</code> et émet <code>trip:stale</code> . Il n’alerte pas.
GPS coupé, permission retirée	Le client émet <code>trip:signal</code> avec son motif ; le cercle lit « position indisponible » et le dernier point daté. La chaîne d’échéance continue : perdre le signal n’exempte pas de confirmer.
Réseau coupé	Tampon local borné en Drift, vidé à la reconnexion avec les horodatages réels. Même chorégraphie de reprise que <code>RESUME_ACK_MS</code> (<code>src/socket/state/callState.js</code>).
Deux appareils du propriétaire	Exactement le problème des appels. Un lien (<code>tripId</code> , <code>ownerId</code>) $\rightarrow$ <code>deviceId</code> sur le modèle de <code>src/socket/state/callDeviceOwnership.js</code> . Sans lui, deux appareils entrelacent leurs positions et la trace zigzague.
Trajet plus long que prévu	Prolongations illimitées, plus un plafond dur de 12 h qui force <code>awaiting_confirm</code> . Un trajet n’est pas un traceur permanent.
Compte supprimé ou exclu	Clôture en <code>closed_cancelled</code> , destinataires notifiés.
Horloge du client fausse	Toute l’arithmétique d’échéance est serveur, en UTC, via <code>job_queue.run_after</code> . Le client n’envoie qu’une durée ou une heure ; c’est <code>eta_at</code> calculé par le serveur qui fait foi.

CHAPITRE 8

Modèle de données

8.1 Quatre tables

```
1  -- 051_trips.sql -- trajets de confiance : suivi temps reel, echeance
   et SOS.
2  -- L'audience est figee au demarrage (instantane de la liste
   Confiance),
3  -- sur le modele de broadcast_delivery (040_broadcast.sql).
4
5  CREATE TABLE IF NOT EXISTS trip (
6      id                BIGINT                NOT NULL AUTO_INCREMENT,
7      owner_id          INT                    NOT NULL,
8      client_id         VARCHAR(64)           NOT NULL,          -- creation
   idempotente
9      kind              ENUM('taxi','meeting','sos')              NOT NULL DEFAULT
   'meeting',
10     state              ENUM('active','awaiting_confirm','alert','sos',
   'closed_confirmed','closed_cancelled',
11     'closed_expired','closed_unwatched')
12                                     NOT NULL DEFAULT
13                                     'active',
14     dest_lat           DECIMAL(9,6) NULL,
15     dest_lng           DECIMAL(9,6) NULL,
16     dest_label         VARCHAR(160) NULL,          -- geocode UNE fois
   a la creation
17     dest_radius_m      SMALLINT              NOT NULL DEFAULT 150,
18     eta_at             DATETIME              NULL,
19     grace_minutes      SMALLINT              NOT NULL DEFAULT 10,
20     extensions         SMALLINT              NOT NULL DEFAULT 0,
21     note               VARCHAR(200) NULL,
22     owner_device       VARCHAR(128) NULL,          -- seul appareil
   emetteur
23     last_lat           DECIMAL(9,6) NULL,
24     last_lng           DECIMAL(9,6) NULL,
25     last_acc_m         SMALLINT              NULL,
26     last_battery       TINYINT              NULL,
27     last_at            DATETIME              NULL,
28     stale              TINYINT(1)           NOT NULL DEFAULT 0,
29     started_at         DATETIME              NOT NULL DEFAULT CURRENT_TIMESTAMP,
30     alerted_at         DATETIME              NULL,
31     closed_at          DATETIME              NULL,
32     close_reason       VARCHAR(30)          NULL,
33     points_purged_at   DATETIME              NULL,
34     PRIMARY KEY (id),
35     UNIQUE KEY uq_trip_client (owner_id, client_id),
36     KEY idx_trip_owner   (owner_id, started_at),
37     KEY idx_trip_open    (state, eta_at),
```

```

38     KEY idx_trip_purge (closed_at, points_purged_at),
39     CONSTRAINT fk_trip_owner FOREIGN KEY (owner_id)
40     REFERENCES users(alanyaID) ON DELETE CASCADE
41 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
42
43 CREATE TABLE IF NOT EXISTS trip_watcher (
44     trip_id          BIGINT      NOT NULL,
45     alanyaID         INT         NOT NULL,
46     state            ENUM('active','revoked','left') NOT NULL DEFAULT
47         'active',
48     revoke_reason    VARCHAR(20) NULL,           -- blocked / removed
49         / left
50     msgID            INT         NULL,           -- la carte type 9
51         chez ce membre
52     conversID        INT         NULL,
53     added_at         DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
54     seen_at          DATETIME NULL,
55     revoked_at       DATETIME NULL,
56     PRIMARY KEY (trip_id, alanyaID),
57     KEY idx_tw_user (alanyaID, trip_id),
58     CONSTRAINT fk_tw_trip FOREIGN KEY (trip_id)
59     REFERENCES trip(id) ON DELETE CASCADE,
60     CONSTRAINT fk_tw_user FOREIGN KEY (alanyaID)
61     REFERENCES users(alanyaID) ON DELETE CASCADE
62 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
63
64 CREATE TABLE IF NOT EXISTS trip_point (
65     id                BIGINT      NOT NULL AUTO_INCREMENT,
66     trip_id           BIGINT      NOT NULL,
67     lat               DECIMAL(9,6) NOT NULL,
68     lng               DECIMAL(9,6) NOT NULL,
69     acc_m             SMALLINT    NULL,
70     speed_kmh         SMALLINT    NULL,
71     battery           TINYINT     NULL,
72     recorded_at       DATETIME(3) NOT NULL,       -- horloge client
73     received_at       DATETIME    NOT NULL DEFAULT CURRENT_TIMESTAMP,
74     PRIMARY KEY (id),
75     KEY idx_point_trip (trip_id, recorded_at),
76     CONSTRAINT fk_point_trip FOREIGN KEY (trip_id)
77     REFERENCES trip(id) ON DELETE CASCADE
78 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
79
80 CREATE TABLE IF NOT EXISTS trip_event (
81     id                BIGINT      NOT NULL AUTO_INCREMENT,
82     trip_id           BIGINT      NOT NULL,
83     kind              VARCHAR(24) NOT NULL,
84     actor_id          INT         NULL,           -- NULL = systeme
85     meta              JSON        NULL,
86     created_at        DATETIME    NOT NULL DEFAULT CURRENT_TIMESTAMP,
87     PRIMARY KEY (id),
88     KEY idx_event_trip (trip_id, id),
89     CONSTRAINT fk_event_trip FOREIGN KEY (trip_id)
90     REFERENCES trip(id) ON DELETE CASCADE
91 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
92
93 INSERT IGNORE INTO scheduler_leases (name) VALUES
94     ('trip_nightly_purge');

```

Listing 8.1 – migrations/051_trips.sql — extrait

Les valeurs de `trip_event.kind` : `started`, `extended`, `arrival_detected`, `eta_due`, `confirmed`, `alerted`, `sos`, `resolved`, `watcher_seen`, `watcher_revoked`, `signal_lost`, `signal_back`, `low_battery`, `device_takeover`, `closed`.

Les migrations sont appliquées à la main. Comme le rappelle déjà l'en-tête de `migrations/038_contact_lists.sql`, il n'existe ni exécuteur ni table de suivi. `051_trips.sql` doit être appliqué **avant** de déployer le code qui nomme ces tables. La base est distante : prévoir la fenêtre.

8.2 Pourquoi `DECIMAL(9,6)` et aucun index spatial

Six décimales valent environ 11 cm — très au-delà de ce qu'un GPS de téléphone sait produire. Le type est exact, comparable, se sauvegarde en dump texte sans surprise, et il rend facile de dire honnêtement quelle précision on stocke.

Un type `POINT` et un index `SPATIAL` n'apporteraient rien : on ne cherche **jamais** de proximité entre trajets. L'arrivée s'évalue par trajet, contre une destination unique, en mémoire, à l'ingestion du point. Les cinquante migrations existantes n'ont aucun index spatial ; en introduire un pour un calcul qu'on ne fait pas serait un coût d'exploitation gratuit.

8.3 L'instantané de l'audience

`trip_watcher` est peuplé **une fois**, au démarrage, à partir de *tous* les membres de la liste `kind = 'trust'` du propriétaire. C'est le même geste que `broadcast_delivery` (`migrations/040_broadcast.sql`) : matérialiser l'audience pour pouvoir dire, après coup, qui a été destinataire.

Trois raisons :

1. un trajet est une pièce d'archive — si l'alerte part, il faut pouvoir répondre à « qui a été prévenu ? » ;
2. les modifications de `contact_list_member` ne doivent pas se propager à un trajet en cours, sinon l'audience devient indécidable ;
3. cela neutralise la faille décrite plus bas : supprimer sa liste Confiance pendant un trajet n'ampute pas le trajet en cours.

Pas de sous-sélection — le cercle est l'audience. Le composeur ne propose aucune case à cocher : démarrer un trajet notifie les cinq membres du cercle, en bloc. C'est ce qui permet de partir en un appui, et c'est ce qui rend la promesse énonçable en une phrase.

Le corollaire est que l'audience se choisit **hors trajet**, en modifiant la liste Confiance — un geste à froid, délibéré, et qui ne notifie personne, conformément à la règle de confidentialité posée au volet 2. Au moment de partir, il n'y a plus rien à décider ; c'est précisément l'objectif.

Une seule exception en cours de route, à l'échelle du trajet et sans effet sur la liste : le propriétaire peut **révoquer** un destinataire. C'est aussi le mécanisme qu'emprunte la révocation automatique en cas de blocage.

Deux défauts connus des listes deviennent des défauts de sûreté. `deleteList` (`src/controllers/contactListController.js`) protège le *nom* des listes système mais pas leur *existence* : supprimer « Confiance » réussit, et `ensureDefaultContactLists` la recrée **vide** au prochain `GET /api/contact-lists`. Aujourd'hui on perd un rangement ; demain on vide un cercle de sécurité sans le dire.

`getListMembers` appelle `maskPresenceIfBlocked` par ligne, mais **n'exclut pas** les membres qui ont bloqué le propriétaire. Le composeur de trajet proposerait donc de confier sa sécurité à quelqu'un qui l'a bloqué.

Les deux figurent déjà aux points ouverts du volet 2. Ils deviennent des **prérequis du même lot**.

8.4 Mémoire et base

La règle de partage. Un tic de position peut se perdre. Une transition d'état, non.

En mémoire, dans un `src/socket/state/tripState.js` calqué sur `callState.js` : dernière position, horodatage du dernier envoi, compteur de décimation, ensemble des destinataires, ensemble des blocages, minuterie de péremption, cache du lien d'appareil.

En base, tout ce qui doit survivre à un redémarrage. C'est la différence majeure avec les appels : **un trajet survit au redémarrage du serveur**. La base fait foi ; la mémoire n'est qu'un cache et un limiteur de débit, reconstruit paresseusement au premier contact.

CHAPITRE 9

Le fil des positions

9.1 Trois régimes, pilotés par la distance

Le moteur est `Geolocator.getPositionStream()` et son `distanceFilter`, pas une minuterie. Le système d'exploitation pousse un point *quand l'appareil a bougé* ; à l'arrêt, il n'émet rien.

Régime	Déclencheur	Précision	Filtre	Plancher	Battement
Nominal	à plus de 10 min de l'ETA	balanced	25 m	5 s	45 s
Approche	à moins de 10 min, ou 1 km du but	high	15 m	4 s	20 s
Alerte	états alert et sos	best	15 m	5 s	15 s

Le battement forcé est indispensable : sans lui, on ne distingue pas *immobile* de *traceur mort*. C'est exactement l'information dont un destinataire a besoin. Il ne rallume pas le GPS — il renvoie le dernier point connu (`Geolocator.getLastKnownPosition()`, déjà utilisé comme repli dans `lib/screens/chats/location_picker_screen.dart`) avec son horodatage de capture réel. L'écart entre `recorded_at` et `received_at` dit alors, à lui seul, si l'appareil mesure encore.

Le `distanceFilter` seul ne borne pas le débit. Un filtre de 50 m produit un point toutes les 36 s à pied, mais toutes les **6 secondes** dans un taxi à 30 km/h, et toutes les 2 secondes sur voie rapide. Le débit est donc proportionnel à la vitesse — et le taxi, qui est le cas d'usage principal, est le plus coûteux.

D'où le **plancher** de la quatrième colonne : deux envois ne peuvent pas être séparés de moins de n secondes. Les points intermédiaires ne sont pas jetés, ils sont *groupés* dans l'envoi suivant — la trace garde sa finesse, seul le nombre de paquets est borné.

9.2 Aucune de ces valeurs n'est écrite en dur

Ce sont des **points de départ à régler en service**, pas des constantes. Deux conséquences de conception, et la seconde est la moins évidente.

Le serveur en est propriétaire. Un module `src/constants/tripPolicy.js` lit l'environnement et expose la politique. Il n'existe pas de table de réglages globale dans Alanya (`appSettingsService.js` est *par utilisateur*) ; la convention maison est la variable d'environnement, comme `NOTIFICATION_ANDROID_NATIVE_V2`.

La politique doit être *servie*, jamais compilée dans l'application. La cadence s'applique sur le téléphone. Si ces valeurs vivent dans des constantes Dart, les changer impose une **publication de l'application** et des semaines d'adoption — autant dire qu'elles ne se règlent jamais.

POST /api/trips et **GET** /api/trips/active renvoient donc un objet **policy**, que le client applique. Les constantes Dart ne servent que de repli, si la réponse est absente ou le serveur plus ancien. `trip:state` la reporte aussi, ce qui permet à un réglage de prendre effet **sur un trajet déjà en cours**.

Variable	Défaut	Ce que coûte un réglage plus large
TRIP_FILTER_M_NOMINAL	25 m	Trace plus anguleuse en ville ; sans effet sur la sûreté.
TRIP_FLOOR_S_NOMINAL	5 s	Le pin saccade. C'est la molette la plus sûre à tourner : elle ne change que le nombre de paquets.
TRIP_BEAT_S_NOMINAL	45 s	Un arrêt met plus longtemps à se confirmer, et le seuil de péremption recule d'autant.
TRIP_FILTER_M_APPROACH	15 m	L'arrivée se détecte plus tard.
TRIP_FLOOR_S_APPROACH	4 s	—
TRIP_BEAT_S_APPROACH	20 s	—
TRIP_FILTER_M_ALERT	15 m	À ne pas relâcher — c'est le seul régime où la précision sert vraiment.
TRIP_FLOOR_S_ALERT	5 s	À ne pas relâcher.
TRIP_BEAT_S_ALERT	15 s	À ne pas relâcher.
TRIP_STALE_FACTOR	3	Seuil de péremption = facteur × battement + 30 s.
TRIP_BROADCAST_MIN_S	5 s	Cadence de rediffusion aux proches.
TRIP_PERSIST_MIN_S	30 s	Finesse de la trace rejouable, et volume de <code>trip_point</code> .
TRIP_LOW_BATTERY_PCT	15 %	Seuil de bascule en suivi ralenti.

Deux familles de réglages, qui ne se propagent pas pareil. Les valeurs de **cadence** ci-dessus sont pures dépenses : elles s'appliquent à chaud, y compris sur un trajet en cours.

Les valeurs de **contrat** — `dest_radius_m`, `grace_minutes`, les relances, la durée maximale — sont d'une autre nature : elles définissent ce qui a été promis à l'utilisateur au moment du départ. Elles sont donc **recopiées dans la ligne `trip`** à la création (les deux premières y sont déjà des colonnes), et un réglage ultérieur ne modifie **jamais** un trajet déjà parti. Sinon on changerait le contrat après signature.

Garde batterie. Sous 15 %, retour au régime nominal avec un battement de 120 s, et le cercle est *informé* (« batterie faible — suivi ralenti »). Sous 5 %, émission d'un événement portant la dernière position : un téléphone qui meurt est un signal utile, et il ne doit pas être confondu avec une disparition.

9.3 Décimation à trois étages

1. **Client** — le `distanceFilter` supprime déjà tout ce qui n'est pas un déplacement.
2. **Socket** — le serveur ne rediffuse pas plus d'une position toutes les 5 s par trajet, quelle que soit la cadence entrante.

3. **Base** — une insertion dans `trip_point` toutes les 30 s au plus, par lots de 10 s. La dernière position vit dans les colonnes `last_*` de `trip`, qui sont écrasées ; la table `trip_point` n'est là que pour rejouer la trace.

9.4 Détection d'arrivée

Trois conditions cumulatives, et une conséquence :

1. être à moins de `dest_radius_m` (150 m par défaut) ;
2. y rester **60 s** — l'hystérésis ;
3. se déplacer à moins de ~ 15 km/h — le portillon de vitesse.

Les relevés dont `acc_m` > 100 sont **ignorés pour la détection**, mais affichés quand même, grisés, avec leur cercle de précision. Mieux vaut dire « à 60 m près » que planter un point mensonger.

Pourquoi 150 m, et pas 50. En ville, la précision courante est de 20 à 65 m, et le multi-trajet en canyon urbain dépasse régulièrement 100 m. Un rayon de 50 m ne déclencherait jamais en centre-ville ; 500 m déclencherait à un pâté de maisons. L'hystérésis et le portillon de vitesse font le reste du travail : passer devant sa rue sans s'arrêter est le faux positif numéro un.

Et la conséquence, déjà énoncée mais qui mérite d'être répétée ici : **une arrivée détectée ne clôt rien**. Elle fait passer en `awaiting_confirm` et pose la question.

9.5 Coût réseau et charge serveur

Une position pèse environ 130 octets de JSON, et ~ 220 octets sur le fil une fois l'enveloppe Socket.IO, WebSocket, TLS et TCP/IP comptée. Ce n'est **pas** un appel HTTP : la trame part sur la socket déjà ouverte — celle des messages, de la présence et des appels — sans en-tête, sans jeton à revalider, et **sans réponse attendue**.

C'est la capture qui suit la vitesse ; le plancher, lui, borne les envois :

Allure	Captures/h	Envois/h	Montant, trajet de 30 min
5 km/h, à pied	100	100	~ 11 ko
30 km/h, taxi ville	600	240	~ 26 ko
90 km/h, voie rapide	1 800	240	~ 26 ko

Le plancher de 15 s plafonne les envois à 240 par heure quelle que soit l'allure — sans lui, un taxi en ville en produirait 600, et le double sur voie rapide.

En descendant, un proche *dont l'écran est ouvert* reçoit au plus une trame toutes les 5 s, soit ~ 160 ko/h. En pratique il ne regarde presque jamais : il reçoit la notification et ouvre la carte une ou deux fois.

Le vrai poste de dépense est la carte, pas les positions. Une session de suivi de cinq minutes avec une carte qui se déplace charge 60 à 150 tuiles, soit **1 à 3 Mo**. Les tuiles coûtent **20 à 50 fois** plus cher que le flux de positions qu'elles servent à afficher.

C'est ce qui justifie deux choix du dossier : la carte du proche est une *vignette* figée dans la conversation, jamais une carte vivante ; et un fournisseur de tuiles sous contrat est un prérequis de mise en service.

À 1 000 trajets simultanés	Ordre de grandeur
Trames socket entrantes	~70 / s — aucune connexion nouvelle, elles empruntent les sockets existantes
Émissions sortantes	~100 à 200 / s
Insertions <code>trip_point</code>	33 lignes/s, groupées en un INSERT multi-lignes toutes les 2 s
Stockage	1 000 trajets de 30 min = 60 000 lignes $\approx$ 6 Mo, purgés à 24 h
Lignes de <code>job_queue</code>	4 000 armées

Pour situer cet ordre de grandeur dans l'existant : `typing:start` est aujourd'hui émis à **chaque caractère frappé**, et coûte « 1+2N requêtes SQL par frappe » (`docs/AUDIT_SCALABILITE_2026-08-06.md`). Une seule conversation de groupe animée coûte plus de SQL que cent trajets.

Le plafond réel de la file est de 20 jobs par seconde. `startJobWorker()` tourne à `setInterval(tick, 1000)` avec `BATCH_SIZE = 20` (`src/services/jobQueue.js`). Aux heures où les ETA s'agglutinent — 18h-20h — c'est juste. `idx_job_pret` et `FOR UPDATE SKIP LOCKED` tiennent la concurrence ; c'est `BATCH_SIZE` qui est la molette. À mesurer avant la montée en charge, pas après.

Batterie — le poste qui compte vraiment. `balanced` coûte 3 à 6 %/h, `high` 8 à 15 %/h, `best` en continu 15 à 25 %/h. Un taxi de 30 minutes en régime nominal revient à 2 ou 3 %. C'est le régime d'alerte qui est cher — et c'est une des raisons du plafond de durée.

CHAPITRE 10

L'échéance et l'escalade

10.1 L'escalade en trois temps

Moment	Job	Effet
$T - 5 \text{ min}$	<code>trip_eta_soon</code>	Silencieux, propriétaire seul : « vous arrivez bientôt ? Prolonger / Je suis arrivé ».
T	<code>trip_eta_due</code>	État <code>awaiting_confirm</code> , notification prioritaire au propriétaire seul ; relances à +0, +3 et +7 min.
$T + 10 \text{ min}$	<code>trip_grace_expired</code>	État <code>alert</code> : le cercle est prévenu, avec la dernière position connue.

Les cinq types de notification poussée. Chaque étape de l'escalade porte son propre type, et chacun compose son *titre* et son *corps* côté serveur : les envois sont *data-only*, et quand l'application est tuée c'est le système qui affiche — il n'y a personne pour traduire. Une charge utile sans ces deux champs produit une notification parfaitement silencieuse.

`trip_eta_soon`, `trip_due` et `trip_reminder` vont au **propriétaire seul**, sur le canal `alanya_trip`, en priorité haute mais **sans** traverser « Ne pas déranger » : ce sont des rappels, pas des alertes.

`trip_alert` et `trip_sos` vont au cercle, sur le canal `alanya_trip_alert`, dont l'usage audio est déclaré `alarm`. C'est ce qui leur fait passer la catégorie « alarmes » de « Ne pas déranger », autorisée par défaut — la seule voie sans `ACCESS_NOTIFICATION_POLICY`, permission que l'utilisateur doit accorder à la main. Sur iOS, `InterruptionLevel.timeSensitive` joue le même rôle face au mode Concentration. Ce sont les deux seuls types de toute l'application autorisés à traverser le silence.

Le *tag* de notification dérive du trajet et non du type : un trajet occupe une ligne dans le tiroir, et le rappel d'échéance remplace le pré-avis au lieu de s'empiler à côté. Même principe que la carte de conversation, réécrite sur place.

Pourquoi dix minutes, et non cinq. En deçà de cinq minutes, chaque embouteillage devient une alerte, et le cercle apprend à ignorer les alertes. C'est le véritable mode de défaillance de ce genre de produit : pas l'alerte manquée, mais l'alerte qu'on n'ouvre plus. Au-delà d'un quart d'heure, le délai n'a plus d'utilité dans un incident réel.

La **prolongation** est à deux appuis, +15 ou +30 minutes, sans limite de nombre. Chaque prolongation ré-arme les jobs et écrit un `trip_event` visible du cercle (« Awa a prolongé de 15 minutes ») : cela rassure sans alerter.

10.2 Les huit `kind` de jobs

`trip_eta_soon`, `trip_eta_due`, `trip_reminder`, `trip_grace_expired`, `trip_max_duration`, `trip_no_watcher`, `trip_stale_check`, `trip_points_purge`. Un service `src/services/tripWorkers.js` les enregistre, sur le modèle de `src/services/broadcastWorkers.js`.

`dedupe_key = "trip-<id>"` pour chacun. La contrainte `UNIQUE uq_job_dedupe (kind, dedupe_key)`

de migrations/039_job_queue.sql rend le ré-armement trivial : une transaction qui supprime les lignes du trajet puis appelle `enqueue` avec le nouveau `run_after`.

enqueue est silencieux sur collision. `enqueue` fait `ON DUPLICATE KEY UPDATE id = id` et **renvoie `null`** si la clé de déduplication existe déjà. Prolonger un trajet sans supprimer d'abord la ligne existante *paraît* fonctionner et laisse l'ancienne échéance armée — l'alerte partirait à l'heure initiale. Le `DELETE` préalable n'est pas une optimisation, c'est la correction.

La file est *at-least-once*. Un job peut se déclencher après la clôture du trajet — reprise après incident, orphelin récupéré par `reclaimOrphans` au bout de dix minutes. **Chaque handler relit `trip.state` et sort en silence si le trajet est terminal.** C'est la seule protection qui tienne.

La purge nocturne tourne sous le bail `trip_nightly_purge` via `withLease()` (`src/services/schedulerLease.js`), à côté de `broadcast_nightly_purge` et `welcome_status_purge`.

CHAPITRE 11

API

11.1 REST — `src/routes/trips.js`

Style copié de `contactListController.js` : cadrage sur `req.user.alanyaID`, jamais sur un identifiant fourni par le client ; `pool.execute` ; codes 400/403/404/409.

POST <code>/api/trips</code>	Démarrer. {clientId, kind, dest?, etaAt? durationMin?, note?} — l'audience n'est pas un paramètre
POST <code>/api/trips/sos</code>	SOS autonome — crée le trajet
GET <code>/api/trips/active</code>	Reprise à froid : mon trajet, et ceux que je suis
GET <code>/api/trips/:id</code>	Détail et frise d'événements
GET <code>/api/trips/:id/points?since=</code>	La trace — vide après purge
POST <code>/api/trips/:id/extend</code>	{minutes}, ré-arme les jobs
POST <code>/api/trips/:id/confirm</code>	Arrivée confirmée
POST <code>/api/trips/:id/cancel</code>	Arrêt avant échéance
POST <code>/api/trips/:id/sos</code>	Escalade d'un trajet existant
DELETE <code>/api/trips/:id/watchers/:id</code>	Révoquer un destinataire
POST <code>/api/trips/:id/seen</code>	Acquittement d'un destinataire
GET <code>/api/trips/history?cursor=</code>	Historique du propriétaire
DELETE <code>/api/trips/:id</code>	Suppression par le propriétaire

Codes d'erreur : `TRUST_LIST_EMPTY` 409, `TRIP_ALREADY_ACTIVE` 409, `TRIP_TERMINAL` 409, `TRIP_INCIDENT_LOCKED` 423, `DEVICE_NOT_OWNER` 403.

L'audience n'étant pas transmise par le client, elle n'a pas de code d'erreur propre : le serveur lit la liste `trust` du porteur du jeton, et refuse en 409 `TRUST_LIST_EMPTY` si elle est vide.

Toute réponse portant un trajet porte aussi la politique. **POST** `/api/trips`, **GET** `/api/trips/active` et l'événement `trip:state` renvoient un objet `policy` : les trois régimes, avec leurs filtres, planchers et battements, tels que `tripPolicy.js` les expose au moment de l'appel. Le client l'applique ; ses constantes Dart ne servent que si le champ manque. C'est la seule façon de régler la cadence sans publier l'application. Le champ doit donc exister **dès la première version**, même s'il ne renvoie d'abord que les valeurs par défaut — l'ajouter plus tard imposerait exactement la publication qu'on cherche à éviter.

Aucune position ne passe par REST en régime normal. Un aller-retour HTTP tous les 50 mètres serait absurde. Les positions montent par socket ; **GET** `/api/trips/:id/points` ne sert qu'à rejouer une trace, et le lot de secours n'existe que pour vider le tampon hors ligne.

11.2 Socket.IO — `src/socket/handlers/trips.js`

Enregistré à côté de `calls.js` et `meetings.js` dans `server.js`. Room `trip_<id>`, rejointe après vérification d'appartenance, exactement comme `join_conversation` vérifie `conv_participants`.

Client → serveur	Objet
<code>trip:subscribe</code>	Autorise, puis renvoie l'état complet
<code>trip:unsubscribe</code>	—
<code>trip:position</code>	Un point. Rejeté si <code>deviceId</code> n'est pas <code>trip.owner_device</code>
<code>trip:position_batch</code>	Vidange du tampon hors ligne, 200 points au plus
<code>trip:claim_device</code>	Reprendre la main depuis un autre appareil
<code>trip:signal</code>	<code>gps_off</code> , <code>permission_denied</code> , <code>background_killed</code> , <code>battery_saver</code>
<code>trip:seen</code>	Acquittement d'un destinataire

Serveur → client	Objet
<code>trip:state</code>	État complet — à la souscription et à chaque transition
<code>trip:position</code>	Fan-out room, plafonné à une émission / 5 s / trajet
<code>trip:card_update</code>	Mutation de la carte de type 9
<code>trip:alert</code>	Le trajet et sa dernière position
<code>trip:stale</code>	Plus de position reçue
<code>trip:signal</code>	Relais du motif déclaré par le propriétaire
<code>trip:device_revoked</code>	Un autre appareil a repris la main
<code>trip:closed</code>	Fin, avec son motif

La room ne porte que le mouvement — jamais l'alerte. `trip_<id>` sert **uniquement** au flux haute fréquence. Tout ce qui doit *atteindre* un destinataire — état, alerte, clôture — passe par `emitToUser` (`src/utils/userSocketRegistry.js`) **plus** la notification poussée. Un destinataire dont aucun appareil n'est abonné à la room doit quand même recevoir l'alerte. **Une alerte ne transite jamais par la seule room.**

`isUserOnline()` et `hasForegroundSocket()` décident si un destinataire reçoit le fan-out socket ou seulement la notification : on ne diffuse pas dans une room que personne ne regarde.

11.3 Le verrou d'appareil

`trip.owner_device` porte l'identifiant de l'unique appareil autorisé à émettre. Les autres appareils du compte affichent « trajet en cours sur votre autre appareil » et peuvent reprendre la main explicitement : `trip:claim_device` bascule la colonne, l'ancien porteur reçoit `trip:device_revoked` et arrête son service.

C'est le motif de `src/socket/state/callDeviceOwnership.js`, transposé — avec la même justification qu'aux appels : sans lui, deux appareils entrelacent leurs positions et la trace zigzague entre deux endroits.

CHAPITRE 12

Le message de type 9

12.1 Pourquoi un type de message

Les types existants vont de 0 à 8 ; le 9 est libre. Ils ne sont d'ailleurs documentés nulle part au complet : le commentaire de `LocalMessages.type` (`lib/core/db/app_database.dart`) s'arrête à 5=localisation, le 6 et le 8 sont des constantes nommées (`kSystemMessageType` dans `lib/core/``utils/system_event_payload.dart`, `kWelcomeCtaMessageType` dans `welcome_cta_payload.dart`), et le 7 — la carte de contact — n'existe qu'en littéral, dans `chat_bubbles.dart` et `talky_models.dart`. Le type 9 mérite une constante nommée et un commentaire tenu à jour.

Le choisir plutôt que d'inventer une entité de conversation apporte trois choses gratuitement : la notification poussée, le compteur de non-lus, et la persistance dans l'archive. Et `local_messages` n'a pas à changer — le JSON tient dans `content`, comme pour le type 5.

12.2 L'état dans le message, le mouvement dans le socket

C'est le point technique central du volet.

Cinq écritures, pas cinq mille. Le `content` du message porte l'état : il est réécrit uniquement aux transitions — cinq à six fois sur toute la vie d'un trajet. Les *positions* ne touchent jamais la table `message`. Entre deux transitions, la carte s'anime à partir des `trip:position` reçus.

La réécriture passe par `trip:card_update`, et **jamais** par `message:send` : ce dernier remonterait `sentAt`, incrémenterait `unreadCount` et relancerait une notification à chaque changement d'état.

Exception — les incidents. Une alerte ou un SOS écrit *en plus* un message système de type 6, pour que l'incident reste dans l'archive même si la carte est refermée ou le trajet purgé. Deux lignes au maximum par trajet, dans le pire cas.

12.3 Compatibilité descendante

Un client ancien afficherait « Média ». Les replis existent des deux côtés — `default:` dans `_buildMedia` (`lib/screens/chats/chat/chat_bubbles.dart`) et `default: return mediaName ?? 'Média'` dans `messagePreview` (`src/Utils/messagePreview.js`) — mais ils rendent un libellé générique et, dans le pire cas, le JSON brut. Le rendu dégradé (« Trajet de confiance — mettez à jour l'application ») et l'aperçu de conversation doivent être écrits dans les deux fichiers **avant** la première émission d'un type 9 en production.

L'aperçu suit le patron de `locationPreviewFromContent` : un court-circuit `if (t === 9)` dans `messagePreview`, et une entrée dans `mediaTypeLabel`. Le commentaire existant du fichier vaut aussi ici — *ne jamais exposer le content brut*.

CHAPITRE 13

Notifications

Nouveaux types dans `src/notifications/notificationContract.js` : `trip_started`, `trip_alert`, `trip_sos`, `trip_closed`, `trip_watcher_seen`, et un constructeur `buildTripPayload` en `schemaVersion: '2'` portant `tripId`, `state`, `ownerName`, `lastLat` / `lastLng` / `lastAt` et un lien profond.

Type	Priorité	TTL	Rendu
<code>trip_started</code>	normale	1 h	Notification ordinaire
<code>trip_alert</code>	haute	120 s	<i>data-only</i> , plein écran
<code>trip_sos</code>	haute	120 s	<i>data-only</i> , plein écran, son propre
<code>trip_closed</code>	basse	1 h	Silencieuse
<code>trip_watcher_seen</code>	basse	1 h	Silencieuse, propriétaire seul

Le TTL court des alertes est délibéré : une alerte de sûreté vieille de trois heures est du bruit, et son affichage tardif décrédibilise les suivantes. Le modèle est `sendCallToUser` / `buildIncomingCallPayload`, déjà en `ttl: 45_000`.

Contournement de « Ne pas déranger » — pour deux types seulement. `trip_alert` et `trip_sos` passent outre les préférences de notification et le mode « Ne pas déranger ». Une alerte de sûreté qu'un réglage de silence peut étouffer n'est pas une alerte de sûreté. Le contournement s'arrête là : `trip_started` est une notification ordinaire et `trip_closed` est silencieuse. À valider explicitement — il engage le produit.

Côté client, un canal Android dédié `alanya_trip_alert` en importance haute, avec son propre son et un `fullScreenIntent` pour le SOS ; le canal d'appel (`alanya_call_media`) donne le patron.

CHAPITRE 14

Client Flutter

14.1 Cache local — Drift 23 → 24

`schemaVersion` vaut aujourd'hui **23** (`lib/core/db/app_database.dart`), les listes de contacts l'ayant porté là en trois étapes. Les trajets prennent le **24** :

```
if (from < 24) {  
  await m.createTable(localTrips);  
  await m.createTable(localTripPoints);  
  await m.createTable(localTripEvents);  
}
```

Pure création de tables, aucune migration de données. `local_messages` ne change pas. Puis `dart run build_runner build --delete-conflicting-outputs`. La montée doit passer par `onUpgrade`, jamais par une recréation.

`LocalTripPoints` est un **anneau borné** : tampon hors ligne côté propriétaire, cache de polyligne côté destinataire. Le plafonnement (suppression des plus anciens au-delà de N par trajet) vit dans le DAO, pas dans le schéma.

Deux nouveaux services : `lib/core/services/trip_repository.dart` (REST et Drift) et `lib/core/services/trip_socket_service.dart`. Les deux tables sont à ajouter à `clearSession()`.

14.2 Le service de localisation en avant-plan (Android)

`TripLocationForegroundService.kt`, cloné de `CallMediaForegroundService.kt` : canal `alanya_trip_location`, identifiant de notification distinct, et le même conditionnement de `foregroundServiceType` au SDK 34 que l'existant. Pont `TripLocationBridge.kt` sur le canal `com.alanya237.alanya/trip_location`, calqué sur `CallMediaBridge.kt`.

START_STICKY, et non START_NOT_STICKY. Le service d'appel est délibérément non collant : un appel tué ne doit pas ressusciter. Un trajet, si. Le service relit le trajet ouvert dans Drift au redémarrage et reprend l'émission.

Orchestrateur Dart `lib/core/services/trip_session_guard.dart`, cloné de `lib/core/services/call_session_guard.dart` : singleton, compteur de références, `WidgetsBindingObserver`, démarrage et arrêt du service, propriété de l'abonnement `getPositionStream()`, application des trois régimes, gestion du tampon hors ligne.

Le contenu de la notification persistante a une portée éthique. Elle doit **nommer les destinataires** (« Partagé avec Maman, Karim ») et offrir un « Arrêter » en un appui. Une notification de suivi qui ne dit pas à qui elle envoie la position, c'est le comportement d'un logiciel espion — et c'est ce qui distingue cette fonctionnalité d'un traceur.

14.3 Permissions à ajouter

Cible	Clé	État actuel
Android	ACCESS_BACKGROUND_LOCATION	absente
Android	FOREGROUND_SERVICE_LOCATION	absente
Android	<service ... foregroundServiceType="location">	absent
iOS	NSLocationAlwaysAndWhenInUse UsageDescription	absente
iOS	UIBackgroundModes: location	absente — la liste vaut audio, remote-notification, voip

Sont déjà là et réutilisables : ACCESS_FINE_LOCATION, ACCESS_COARSE_LOCATION, FOREGROUND_SERVICE, WAKE_LOCK, POST_NOTIFICATIONS, REQUEST_IGNORE_BATTERY_OPTIMIZATIONS.

14.4 iOS — ce qui est garanti, et ce qui ne l'est pas

`allowsBackgroundLocationUpdates = true, showsBackgroundLocationIndicator = true` — **on garde la pastille bleue**, on ne la masque jamais — et `pausesLocationUpdatesAutomatically = false`. Repli sur l'enregistrement des changements significatifs de position.

Ne pas promettre la parité. La garantie iOS est « au mieux, avec péremption explicite ». Ce n'est pas un aveu de faiblesse : c'est la conséquence directe du postulat du chapitre 1. Le filet est la chaîne d'échéance côté serveur, pas le flux de positions — et cette chaîne est identique sur les deux plateformes.

14.5 Interface

Fichier	Objet
<code>screens/trips/ trips_hub_screen.dart</code> (<i>nouveau</i>)	Démarrer, trajet en cours, historique.
<code>screens/trips/trip_compose_ screen.dart</code> (<i>nouveau</i>)	Type, destination, heure, destinataires, consentement.
<code>screens/trips/ trip_live_screen.dart</code> (<i>nouveau</i>)	Suivi — vue propriétaire et vue destinataire.
<code>widgets/chat/ trip_message_card.dart</code> (<i>nouveau</i>)	La carte de type 9, huit états.
<code>widgets/trips/trip_banner.dart</code> (<i>nouveau</i>)	Bandeau persistant au-dessus de la barre de navigation.
<code>screens/profile/ profile_screen.dart</code>	Entrée « Trajets de confiance », à côté des listes.
<code>screens/chats/chat/ chat_actions.dart</code>	Le menu position devient un choix à deux entrées.
<code>lib/l10n/*</code>	Libellés des états, de l'escalade et du consentement.

Aucun sixième onglet — et pas besoin. `lib/screens/home/glass_nav_bar.dart` construit sa rangée par `List.generate(5, ...)` : Discussions, Appels, Statuts, Réunions, Profil. Le bandeau persistant se pose *au-dessus* de la barre, dans l'espace déjà réservé par `kGlassNavBarSpace`. Il est visible sur les cinq onglets : c'est le sixième onglet sans en être un.

Les tuiles cartographiques ne sont pas gratuites à cette échelle. `location_picker_screen.dart` charge `tile.openstreetmap.org` une fois, pour un écran. Un trajet de trente minutes suivi par cinq personnes en charge des *centaines*. La politique d'usage d'OpenStreetMap l'interdit explicitement pour ce volume. Un fournisseur sous contrat est un **prérequis de mise en production**, pas une optimisation ultérieure. Le composant `flutter_map` ne change pas — seule l'URL des tuiles change.

CHAPITRE 15

Vie privée, coercition et administration

15.1 Base légale et minimisation

La base légale est le **consentement**, donné par trajet, révocable à tout instant en arrêtant le partage. Pas l'intérêt légitime : une trace de déplacement continue liée à une identité révèle par inférence la pratique religieuse, l'état de santé et l'orientation sexuelle. Elle n'est pas une donnée personnelle ordinaire.

Le géocodage inverse passe par `nominatim.openstreetmap.org (lib/core/utils/location_payload.dart)` — un tiers. On géocode donc **la destination une seule fois**, à la création, et **jamais la trace**.

15.2 Rétention — le tableau fait foi

Donnée	Trajet normal	Trajet à incident	Raison
trip_point	24 h après clôture	30 jours	Un historique permanent des déplacements de tous les utilisateurs n'est justifié par aucun besoin produit. En cas d'incident, la trace peut être une preuve.
trip	12 mois	12 mois	L'historique produit se contente du fait.
trip_event	12 mois	12 mois	Source de la frise.

La suppression par l'utilisateur est immédiate pour un trajet normal, avec effacement de la carte chez les destinataires.

Le verrou de 30 jours sur les incidents est une mesure anti-coercition. Un trajet clos en **alert** ou **sos** ne peut pas être supprimé avant 30 jours (423 TRIP_INCIDENT_LOCKED). C'est délibéré : un agresseur ne doit pas pouvoir contraindre quelqu'un à effacer la trace. À expliquer dans l'interface, pas à subir.

La suppression de compte doit être complétée dans `src/services/accountDeletionService.js` : les `ON DELETE CASCADE` couvrent l'essentiel, la vérification explicite reste due.

15.3 Ce que l'administration voit — et ce qu'on refuse de construire

L'administration reçoit une page de **compteurs agrégés** et rien d'autre : trajets démarrés, clos, en alerte par jour ; durée médiane ; taux d'alerte ; taux de SOS. Sans `alanyaID`, sans coordonnées, sans libellé de destination, sans identité de destinataire. La coque `components/geolocation/` de l'administration `Next.js` (`WorldMap.tsx`, `GeoStats.tsx`) se réutilise telle quelle pour cette vue.

Deux options examinées, puis écartées. Un **registre d'incidents pseudonymisé** (qui, quand, quelle issue) aiderait le support ; un **bris de glace** sur la dernière position, à quatre yeux, avec justification écrite, expiration à 15 minutes, journal en ajout seul et notification à la personne concernée, aiderait en cas d'incident réel.

Les deux sont écartés pour cette étape. Non parce qu'ils seraient mal conçus, mais parce que la posture la plus défendable est celle où **la donnée n'est pas accessible**. Et la rétention l'emporte de toute façon sur le contrôle d'accès : à 24 h de conservation hors incident, la fenêtre de consultation n'existe quasiment pas.

Si le besoin revient, la forme est connue — une table `trip_admin_access` en ajout seul, portant `admin_id`, `approver_id`, `reason`, `expires_at` et `subject_notified_at`. Elle n'est pas créée par `051_trips.sql`.

L'AIPD est un prérequis, pas un rattrapage. Un suivi de localisation systématique relève de l'article 35 du RGPD : l'analyse d'impact est **obligatoire**. Elle doit être menée avant la mise en production, et elle doit inclure un modèle de menace « violences conjugales » — pas seulement un modèle de menace technique.

15.4 Détournement et coercition

C'est le risque qui peut tuer la fonctionnalité, et il ne se traite pas par une case à cocher. Sept garde-fous, traités comme des *contraintes de conception* :

1. **Le propriétaire est toujours l'émetteur.** Aucune demande de position, aucune invitation à partager, aucun flux d'acceptation qu'un partenaire puisse transformer en permanence.
2. **Tout trajet est fini et démarré à la main.** Pas de « partager 24 h », pas de récurrence, pas d'interrupteur permanent.
3. **La notification persistante nomme l'audience** et ne peut pas être masquée (service en avant-plan Android, pastille bleue iOS).
4. **Les destinataires n'ont pas d'historique.** On ne peut pas ouvrir « tous les trajets passés d'Awa ». Cela tue l'interrogatoire « où étais-tu mardi ? ».
5. **L'audience se choisit à froid, et en silence.** On ne compose pas son cercle au moment de partir : on l'a fait avant, dans ses listes de contacts, et cette modification ne notifie personne — ni celui qui entre, ni celui qui sort. Sans cela, « tu m'as retiré » deviendrait un levier de pression exerçable en direct.
6. **La sortie est sûre.** « Arrêter le partage » est toujours à un appui, et un arrêt avant échéance n'envoie qu'un message neutre. Si arrêter était visiblement suspect, arrêter deviendrait punissable.
7. **Le verrou de suppression de 30 jours** sur les trajets clos en incident.

Ce sont des atténuations, pas une solution. Aucune de ces mesures n'empêche quelqu'un de contraindre un proche à démarrer un trajet. Elles rendent la contrainte moins rentable et moins durable. Il faut l'écrire tel quel dans l'AIPD : prétendre le contraire serait la plus dangereuse des erreurs de conception.

15.5 Le blocage, et le cercle qui se vide

Le blocage est la primitive de sûreté de l'application ; le respecter n'est pas négociable. Un destinataire qui bloque le propriétaire passe en `state = 'revoked'` avec `revoke_reason = 'blocked'`, quitte la room, voit sa carte effacée, et **ne reçoit plus rien** — **alertes comprises**. Le propriétaire lit « un membre n'est plus destinataire », jamais « X vous a bloqué » : l'application ne révèle pas un blocage.

Le contrôle tourne **à la création** (filtre de l'instantané) **et à chaque diffusion persistée** (alerte, clôture) — pas seulement à la création, car le blocage peut survenir en route et la room ne le sait pas. La requête symétrique existe déjà : `isBlockedEitherWay` (`src/utils/blockUtils.js`), et sa forme groupée dans `src/utils/conversationParticipantsBatch.js`.

S'il ne reste aucun destinataire, le trajet ne continue pas. Un trajet sans destinataire n'est plus une fonctionnalité de sécurité : c'est un traceur personnel, un autre produit, et une responsabilité RGPD qu'on ne veut pas assumer. Le propriétaire reçoit une invite immédiate et sonore ; sans réponse au bout de 5 minutes, le job `trip_no_watcher` clôt en `closed_unwatched` et la purge normale s'applique.

CHAPITRE 16

Impact fichier par fichier

16.1 Backend

Fichier	Nature	Objet
migrations/051_trips.sql	nouveau	Les quatre tables et le bail
src/routes/trips.js	nouveau	Les quatorze routes, Swagger inline
src/controllers/tripController.js	nouveau	Machine à états, autorisations
src/services/tripWorkers.js	nouveau	Les huit kind de jobs
src/constants/tripPolicy.js	nouveau	Rayon, grâce, plafonds, cadences
src/socket/handlers/trips.js	nouveau	Room, positions, verrou d'appareil
src/socket/state/tripState.js	nouveau	Cache et limiteur de débit
server.js	modifié	Montage des routes et du handler
src/utls/messagePreview.js	modifié	Aperçu du type 9
src/services/ notificationService.js	modifié	notifyTripAlert
src/notifications/ notificationContract.js	modifié	buildTripPayload
src/services/jobQueue.js	modifié	JOB_WORKER_ENABLED
src/controllers/ contactListController.js	modifié	Protection de deleteList, exclusion des blo- queurs
src/services/ accountDeletionService.js	modifié	Trajets à la suppression

16.2 Mobile

Fichier	Nature	Objet
lib/core/db/app_database.dart	modifié	Trois tables, <code>schemaVersion</code> 24
lib/core/services/ trip_repository.dart	nouveau	REST et Drift
lib/core/services/trip_socket_ service.dart	nouveau	Événements temps réel
lib/core/services/ trip_session_guard.dart	nouveau	Cycle de vie, régimes GPS
.../kotlin/.../TripLocation ForegroundService.kt	nouveau	Service en avant-plan
.../kotlin/.../ TripLocationBridge.kt	nouveau	Canal de méthodes
lib/screens/trips/	nouveau	Hub, composeur, suivi
lib/widgets/chat/ trip_message_card.dart	nouveau	La carte de type 9
lib/widgets/trips/trip_banner.dart	nouveau	Bandeau persistant
lib/screens/chats/chat/ chat_bubbles.dart	modifié	case 9 et repli
lib/screens/chats/chat/ chat_actions.dart	modifié	Menu position à deux entrées
lib/screens/profile/ profile_screen.dart	modifié	Entrée « Trajets de confiance »
lib/core/services/ push_service.dart	modifié	Canal d'alerte, lien profond
android/app/src/main/ AndroidManifest.xml	modifié	Deux permissions, un service
ios/Runner/Info.plist	modifié	Deux clés
lib/l10n/app_fr.arb, app_en.arb	modifié	Libellés

16.3 Administration

Fichier	Nature	Objet
app/(dashboard)/trips/page.tsx	nouveau	Compteurs agrégés, sans identité
src/routes/admin/trips.js (<i>backend</i>)	nouveau	Une seule route de statistiques

CHAPITRE 17

Vérification

17.1 Backend

Appliquer `051_trips.sql`, vérifier que le worker de jobs tourne, redémarrer, puis :

1. démarrer un trajet → 201 ; en démarrer un second → 409 TRIP_ALREADY_ACTIVE ;
2. démarrer avec une liste Confiance vide → 409 TRUST_LIST_EMPTY ; démarrer avec cinq membres → cinq lignes dans `trip_watcher`, et cinq notifications ;
3. envoyer un tableau `watchers` dans le corps de la requête : il doit être **ignoré**, pas honoré — l'audience se lit en base, jamais dans la requête ;
4. poser un ETA à +2 min, ne rien faire : vérifier le passage en `awaiting_confirm`, les trois relances, puis l'alerte à +10 min ;
5. prolonger de 15 min et vérifier que **l'ancienne échéance n'a pas survécu** dans `job_queue` — c'est le piège de `dedupe_key` ;
6. émettre une position depuis un second appareil → 403 DEVICE_NOT_OWNER ; `trip:claim_device`, puis vérifier que le premier s'est arrêté ;
7. faire bloquer le propriétaire par un destinataire pendant le trajet : il ne doit plus rien recevoir, et le propriétaire ne doit pas apprendre le blocage ;
8. révoquer le dernier destinataire, et vérifier la clôture en `closed_unwatched` après 5 min ;
9. redémarrer le serveur en plein trajet : l'échéance doit toujours partir.

17.2 Client

`flutter pub get` → `dart run build_runner build --delete-conflicting-outputs` → `flutter run`.

1. **Migration 23** → **24** sur un appareil déjà installé : la mise à jour ne doit **pas** vider les données existantes.
2. La carte de type 9 change d'état sans remonter la conversation ni créer de non-lu.
3. **Hors ligne** : réseau coupé pendant 10 minutes, puis rétabli — les points tamponnés remontent avec leurs horodatages réels, sans doublon.
4. Application balayée depuis les récents : le service se relance et l'émission reprend.
5. Notification persistante : elle nomme les destinataires, et « Arrêter » fonctionne en un appui.
6. Client ancien : la carte de type 9 affiche le repli, pas du JSON.

17.3 Recette terrain — ce qui ne se teste que dehors

1. Marcher jusqu'à la destination et vérifier que l'arrivée est détectée — puis **qu'elle ne clôt rien**.
2. Passer à 100 m de la destination sans s'arrêter : aucune détection ne doit se produire (hystérésis et portillon de vitesse).

3. Traverser un tunnel ou un parking souterrain : le cercle doit lire « position indisponible », **pas** une alerte.
4. Un trajet en voiture de 45 minutes, batterie mesurée avant et après.
5. Téléphone en veille, écran éteint, 20 minutes : le flux ne doit pas s'arrêter sur Android ; sur iOS, constater le comportement réel et l'écrire.

17.4 Points ouverts

1. Rétention de 12 mois du fait `trip` — alignée sur la politique générale de conservation du produit ?
2. **Aucune sous-sélection de l'audience** — décision prise, et c'est ce qui rend le départ instantané. Le point à surveiller après la mise en service : les situations où l'on voudrait partager à trois sur cinq (rendez-vous médical, entretien d'embauche). La réponse actuelle est de sortir la personne de son cercle avant de partir, ce qui est un geste lourd. Si l'usage montre que le cas est fréquent, l'ajout d'une sous-sélection ne coûterait rien au modèle de données : `trip_watcher` est déjà une table d'appartenance par trajet.
3. Le contournement de « Ne pas déranger » doit-il être désactivable par le destinataire ? (Recommandation : non, mais l'arbitrage engage le produit.)
4. Les destinataires se voient-ils entre eux pendant un trajet — le nombre seulement, ou les identités ? (Recommandation : le nombre. Les identités exposent le carnet d'adresses, mais faciliteraient la coordination en cas de SOS.)
5. Qui porte l'AIPD, et bloque-t-elle la mise en production ? (Recommandation : oui.)
6. Le verrou de suppression de 30 jours est-il tenable dans toutes les juridictions visées ?
7. `BROADCAST_WORKER_ENABLED` → `JOB_WORKER_ENABLED` : renommage acceptable, ou faut-il un second drapeau ?
8. Quel fournisseur de tuiles cartographiques, et à quel coût par trajet ?
9. Extensions volontairement hors périmètre, possibles sans changer les tables : déclenchement matériel du SOS, appel automatique des secours, partage à un non-contact par lien.